SEMT Data Science and Applied Statics

Xile Gong - 2630541

Part A -Data Science

Code

Question1

Set up:

In [167…

```
import os
import sys

import numpy as np
import matplotlib.pyplot as plt

from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import KFold
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.cluster import KMeans
from sklearn.metrics import accuracy_score

#Set a fixed random seed to ensure reproducible results
RANDOM_STATE = 42
#Define the results storage directory to manage output files in one place
RESULTS_DIR = "results"


class Tee:
    """A simple tee class that writes stdout to both the console and a log file."""

    def __init__(self, *files):
        self.files = files

    def write(self, obj):
        for f in self.files:
            f.write(obj)

    def flush(self):
        for f in self.files:
            f.flush()


def load_data(path="image_data.npz"):

    data = np.load(path)
    train_X = data["train_X"]
    train_Y = data["train_Y"].ravel()  # Flatten labels to a 1D array
    test_X = data["test_X"]
    test_Y = data["test_Y"].ravel()  # Flatten labels to a 1D array
    return train_X, train_Y, test_X, test_Y
```

Explanation:

Load image data from an .npz file.

Parameters:

-path : str, default="image_data.npz" Path to the .npz file. Return：

-train_X ： Training feature matrix of shape (n_train_samples, n_features).

-train_Y :Training labels as a 1D array.

-test_X :Test feature matrix of shape (n_test_samples, n_features).

-test_Y :Test labels as a 1D array.

Question 1(a):

In [168…

```
def describe_datasets(train_X, train_Y, test_X, test_Y):

    # Get basic dataset information
    n_train, n_features = train_X.shape
    n_test, _ = test_X.shape
    # Assume square images and compute the side length
    image_side = int(np.sqrt(n_features))

    print("=== Question 1(a): Descriptive statistics ===")
    print(f"Train set: {n_train} samples, {n_features} features ({image_side}x{image_side} images)")
    print(f"Test set:  {n_test} samples, {n_features} features ({image_side}x{image_side} images)")

    # Calculate the number of samples per class in the training set
    classes, class_counts = np.unique(train_Y, return_counts=True)
    print(f"Number of classes in training set: {len(classes)}")

    # Plot a bar chart of the class distribution
    plt.figure(figsize=(8, 4))
    plt.bar(classes, class_counts)
    plt.xlabel("Class label")
    plt.ylabel("Number of samples")
    plt.title("Class distribution in training set")
    plt.tight_layout()
    plt.savefig(os.path.join(RESULTS_DIR, "q1a_class_distribution.png"))
    plt.show()
    plt.close()

    return image_side, classes
```

Explanation:

Print basic descriptive statistics and plot the class distribution.

Parameters:

-train_X, train_Y, test_X, test_Y : Training/test features and labels.

Returns:

-image_side : Image side length assuming square images.

-classes : Unique class labels present in the training set.

Question1(b)

In [169…

```
def plot_mean_median_images(train_X, train_Y, image_side, classes):

    print("=== Question 1(b): Mean and median images ===")
    # Use a fixed random seed to ensure reproducibility
    rng = np.random.RandomState(RANDOM_STATE)
    # Randomly select four classes from all classes
    selected_classes = rng.choice(classes, size=4, replace=False)
    print("Selected classes for mean/median images:", selected_classes)

    # Create a 2x4 grid: top row shows mean images, bottom row shows median images
    fig, axes = plt.subplots(2, 4, figsize=(10, 5))

    for idx, c in enumerate(selected_classes):
        # Get all samples of the current class
        class_mask = train_Y == c
        X_c = train_X[class_mask]
        # Compute mean and median images and reshape to 2D image format
        mean_image = X_c.mean(axis=0).reshape(image_side, image_side)
        median_image = np.median(X_c, axis=0).reshape(image_side, image_side)

        ax_mean = axes[0, idx]
        ax_median = axes[1, idx]

        ax_mean.imshow(mean_image, cmap="gray")
        ax_mean.set_title(f"Class {c} mean")
        ax_mean.axis("off")

        ax_median.imshow(median_image, cmap="gray")
```

```
        ax_median.set_title(f"Class {c} median")
        ax_median.axis("off")

    fig.suptitle("Mean and median images for four classes")
    plt.tight_layout()
    fig.savefig(os.path.join(RESULTS_DIR, "q1b_mean_median_images.png"))
    plt.show()
    plt.close(fig)

    return selected_classes
```

Explanation:

Randomly select four classes and plot their mean and median images.

Parameters:

-train_X : Training feature matrix.

-train_Y : Training labels.

-image_side : Image side length.

-classes : Array of all class labels.

Returns:

-selected_classes : The four selected classes.

Question1(c)

In [170…

```
def visualise_two_classes_pca(train_X, train_Y, image_side, two_classes):

    print("=== Question 1(c): 2D visualisation of two classes ===")
    print("Two classes used for 2D visualisation:", two_classes)

    # Create a boolean mask to select samples belonging to the two classes
    mask_two = np.isin(train_Y, two_classes)
    X_two = train_X[mask_two]
    y_two = train_Y[mask_two]

    # Use PCA to reduce the data to two principal components
    pca_2d = PCA(n_components=2, random_state=RANDOM_STATE)
    X_two_2d = pca_2d.fit_transform(X_two)

    plt.figure(figsize=(6, 6))
    markers = ["o", "s"]  # Use different markers to distinguish the two classes
    colors = ["tab:blue", "tab:orange"]  # Use different colours to distinguish the two classes

    # Plot a separate scatter cloud for each class
    for i, c in enumerate(two_classes):
        mask_c = y_two == c
        plt.scatter(
            X_two_2d[mask_c, 0],
            X_two_2d[mask_c, 1],
            label=f"Class {c}",
            alpha=0.5,
            s=10,
            marker=markers[i],
            color=colors[i],
        )

    plt.xlabel("PC1")
    plt.ylabel("PC2")
    plt.title("PCA projection of two selected classes (training set)")
    plt.legend()
    plt.tight_layout()
    plt.savefig(os.path.join(RESULTS_DIR, "q1c_pca_two_classes.png"))
    plt.show()
    plt.close()
```

Explanation:

Project two classes into 2D using PCA and plot the result. Parameters:

-train_X : Training feature matrix.

-train_Y : Training labels.

-image_side : Image side length (kept for API consistency; not required for PCA).

-two_classes : The two class labels to visualise.

Notes:

-This function uses PCA to reduce high-dimensional image data to two principal components to visualise how the two classes are distributed in feature space.

Question1(d.1): Calculate the pairwise distances euclidean

In [171…

```
def pairwise_distances_euclidean(X):

    n_samples = X.shape[0]
    distances = []
    # Iterate over all sample pairs (i, j) with i < j
    for i in range(n_samples):
        for j in range(i + 1, n_samples):
            # Compute the Euclidean distance between sample i and sample j
            d = np.linalg.norm(X[i] - X[j])
            distances.append(d)
    return np.array(distances)
```

Explanation:

Compute all pairwise Euclidean distances between rows of X.

Parameters:

X : Data matrix of shape (n_samples, n_features).

Returns

distances : 1D array of upper-triangular pairwise distances (excluding the diagonal).

Notes

-This function computes distances in the upper triangular matrix (excluding the diagonal) to avoid redundant calculations.

-This implementation is O(n^2) and is intended for small to medium-sized subsets.

Question1(d.2): compare similarity within class and random

In [172…

```
def compare_similarity_within_class_vs_random(train_X, train_Y, label_for_subset):

    print("=== Question 1(d): Similarity within class vs random images ===")
    print(f"Reference class for subset 1: {label_for_subset}")

    # Get all samples of the specified class
    mask_label = train_Y == label_for_subset
    X_label = train_X[mask_label]

    # Set subset size to at most 200 samples
    subset_size = min(200, X_label.shape[0])
    rng = np.random.RandomState(RANDOM_STATE)

    # Subset 1: randomly select samples from the specified class
    indices_label = rng.choice(X_label.shape[0], size=subset_size, replace=False)
    subset1 = X_label[indices_label]

    # Subset 2: randomly select samples from the full training set
    indices_random = rng.choice(train_X.shape[0], size=subset_size, replace=False)
    subset2 = train_X[indices_random]

    print(f"Subset1 (class {label_for_subset}) size: {subset1.shape[0]}")
    print(f"Subset2 (random) size: {subset2.shape[0]}")

    # Calculate within-subset pairwise Euclidean distances for subset 1 and subset 2
    distances_subset1 = pairwise_distances_euclidean(subset1)
    distances_subset2 = pairwise_distances_euclidean(subset2)

    # Plot histograms of the distance distributions for the two subsets
    plt.figure(figsize=(8, 4))
    # Use the same bin range to make the histograms comparable
```

```
    bins = np.linspace(
        min(distances_subset1.min(), distances_subset2.min()),
        max(distances_subset1.max(), distances_subset2.max()),
        30,
    )
    plt.hist(
        distances_subset1,
        bins=bins,
        alpha=0.5,
        label=f"Subset1 (class {label_for_subset})",
    )
    plt.hist(
        distances_subset2,
        bins=bins,
        alpha=0.5,
        label="Subset2 (random images)",
    )
    plt.xlabel("Pairwise Euclidean distance")
    plt.ylabel("Frequency")
    plt.title("Similarity of images: within class vs random")
    plt.legend()
    plt.tight_layout()
    plt.savefig(os.path.join(RESULTS_DIR, "q1d_similarity_histograms.png"))
    plt.show()
    plt.close()
```

Explanation

Create two subsets and compare the within-subset similarity using histograms.

Parameters:

-train_X: feature matrix of the training set

-train_Y: labels of the training set

-label_for_subset: the class label used to create the first subset

Note:

This function creates two subsets

1. subset1: contains only samples from the specified class
2. subset2: randomly selected samples (may come from any class)

By comparing the distribution of pairwise distances within these two subsets, the function analyzes the similarity of samples from the same class.

Question2

Question2(a.1):

In [173…

```
def tune_logistic_regression(X, y, C_values, n_splits=5):

    best_C = None
    best_score = -np.inf
    # Create the K-fold cross-validation splitter
    kf = KFold(n_splits=n_splits, shuffle=True, random_state=RANDOM_STATE)

    # Iterate over all candidate C values
    for C in C_values:
        fold_scores = []
        # Train and validate on each fold
        for train_idx, val_idx in kf.split(X):
            X_tr, X_val = X[train_idx], X[val_idx]
            y_tr, y_val = y[train_idx], y[val_idx]

            # Feature standardisation: fit on the training fold, then transform training and validation folds
            scaler = StandardScaler()
            X_tr_scaled = scaler.fit_transform(X_tr)
            X_val_scaled = scaler.transform(X_val)

            # Train the logistic regression model
            clf = LogisticRegression(C=C, max_iter=1000, random_state=RANDOM_STATE)
```

```
            clf.fit(X_tr_scaled, y_tr)
            y_val_pred = clf.predict(X_val_scaled)
            fold_scores.append(accuracy_score(y_val, y_val_pred))

        # Compute the mean cross-validation score for the current C value
        mean_score = np.mean(fold_scores)
        print(f"Logistic regression: C={C} mean CV accuracy={mean_score:.4f}")

        # Update the best hyperparameter
        if mean_score > best_score:
            best_score = mean_score
            best_C = C

    return best_C, best_score
```

Explanation:

Use K-fold cross-validation to manually tune the hyperparameters of logistic regression.

Parameters:

```
X: feature matrix

y: label vector
C_values: list of regularization parameter values C to test(smaller C means stronger
regularization)

n_splits: number of folds for K-fold cross-validation, default is 5
```

Returns:

```
best_C: the best value of C

best_score: the average cross-validation accuracy corresponding to the best C
```

Note:

```
In logistic regression, C is the inverse of the regularization strength.

A smaller C leads to stronger regularization, which may help prevent overfitting.
```

Question2(a.2):

In [174…

```
def tune_knn_classifier(X, y, k_values, n_splits=5):

    best_k = None
    best_score = -np.inf
    #Create the K-fold cross-validation splitter
    kf = KFold(n_splits=n_splits, shuffle=True, random_state=RANDOM_STATE)

    # Iterate over all candidate k values
    for k_neighbors in k_values:
        fold_scores = []
        #Train and validate on each fold
        for train_idx, val_idx in kf.split(X):
            X_tr, X_val = X[train_idx], X[val_idx]
            y_tr, y_val = y[train_idx], y[val_idx]

            # k-NN is sensitive to feature scales, so standardisation is needed
            scaler = StandardScaler()
            X_tr_scaled = scaler.fit_transform(X_tr)
            X_val_scaled = scaler.transform(X_val)

            # Train the k-NN classifier
            clf = KNeighborsClassifier(n_neighbors=k_neighbors)
            clf.fit(X_tr_scaled, y_tr)
            y_val_pred = clf.predict(X_val_scaled)
            fold_scores.append(accuracy_score(y_val, y_val_pred))

        mean_score = np.mean(fold_scores)
        print(f"k-NN: k={k_neighbors} mean CV accuracy={mean_score:.4f}")

        if mean_score > best_score:
            best_score = mean_score
            best_k = k_neighbors
```

```
    return best_k, best_score
```

Explanation:

Use K-fold cross-validation to manually tune the hyperparameters of k-NN.

Parameters:

```
X: feature matrix

y: label vector

k_values: list of the numbers of neighbors k to test

n_splits: number of folds for K-fold cross-validation, default is 5
```

Returns:

```
best_k: the best value of k (number of neighbors)

best_score: the average cross-validation accuracy corresponding to the best k
```

Note:

```
The k-NN algorithm classifies samples based on their k nearest neighbors.

A smaller k may lead to overfitting, while a larger k may lead to underfitting.
```

Question2(a.3):

In [175…

```
def tune_random_forest_classifier(X, y, n_estimators_list, max_depth_list, n_splits=5):

    best_params = None
    best_score = -np.inf
    # Create the K-fold cross-validation splitter
    kf = KFold(n_splits=n_splits, shuffle=True, random_state=RANDOM_STATE)

    # Grid search: iterate over all parameter combinations
    for n_estimators in n_estimators_list:
        for max_depth in max_depth_list:
            fold_scores = []
            for train_idx, val_idx in kf.split(X):
                X_tr, X_val = X[train_idx], X[val_idx]
                y_tr, y_val = y[train_idx], y[val_idx]

                # Random forests do not require feature normalization because decision trees are based on feature
                clf = RandomForestClassifier(
                    n_estimators=n_estimators,
                    max_depth=max_depth,
                    random_state=RANDOM_STATE,
                    n_jobs=-1,  # Use all available CPU cores
                )
                clf.fit(X_tr, y_tr)
                y_val_pred = clf.predict(X_val)
                fold_scores.append(accuracy_score(y_val, y_val_pred))

            mean_score = np.mean(fold_scores)
            print(
                f"Random forest: n_estimators={n_estimators}, "
                f"max_depth={max_depth} mean CV accuracy={mean_score:.4f}"
            )

            if mean_score > best_score:
                best_score = mean_score
                best_params = (n_estimators, max_depth)

    return best_params, best_score
```

Explanation:

Use K-fold cross-validation to manually tune the hyperparameters of a random forest.

Parameters:

```
X: feature matrix
```

```
y: label vector

n_estimators_list: list of numbers of trees to test

max_depth_list: list of maximum tree depths to test (None means no limit)

n_splits: number of folds for K-fold cross-validation, default is 5
```

Returns:

```
best_params: a tuple of the best parameter combination (n_estimators, max_depth)

best_score: the average cross-validation accuracy for the best parameters
```

note:

```
Random forest is an ensemble learning method that improves predictive performance by combining
multiple decision trees.

n_estimators controls the number of trees, and max_depth controls the depth of each tree.

More trees and deeper depths may improve performance, but they also increase computational cost
and the risk of overfitting.
```

Question2(b)

In [176…

```python
def cluster_purity(y_true, cluster_labels):

    y_true = np.asarray(y_true)
    cluster_labels = np.asarray(cluster_labels)
    assert y_true.shape[0] == cluster_labels.shape[0]

    total = y_true.shape[0]
    purity_sum = 0

    # Iterate over each cluster
    for cluster in np.unique(cluster_labels):
        # Get the true labels of all samples in the current cluster
        mask = cluster_labels == cluster
        true_labels_cluster = y_true[mask]
        if true_labels_cluster.size == 0:
            continue
        # Count samples per true class within the current cluster
        counts = np.bincount(true_labels_cluster)
        # Select the majority class
        majority_label_count = counts.max()
        purity_sum += majority_label_count

    return purity_sum / total
```

Explanation:

Compute clustering purity based on the true labels and the cluster assignments.

Parameters:

```
y_true: array of true class labels

cluster_labels: array of cluster labels assigned by the clustering algorithm
```

Returns:

```
purity: clustering purity score in the range [0, 1]; a larger value indicates better clustering
quality
```

Explanation:

```
Purity is a clustering evaluation metric computed as follows:

1. For each cluster, find the most frequent true class within it.

2. Sum the counts of these majority classes across all clusters.
```

```
3. Divide by the total number of samples to obtain the purity.
```

A purity of 1 indicates perfect clustering, while 1/k (k is the number of classes) indicates random clustering.

Answer

Main function: run the full data analysis and machine learning workflow.

This function includes the following steps:

1. Data loading and descriptive statistics (Question 1a)
2. Mean and median image visualisation (Question 1b)
3. 2D PCA visualisation (Question 1c)
4. Within-class similarity analysis (Question 1d)
5. Classifier training and hyperparameter tuning (Question 2a)
6. K-means clustering analysis (Question 2b)

In [177…

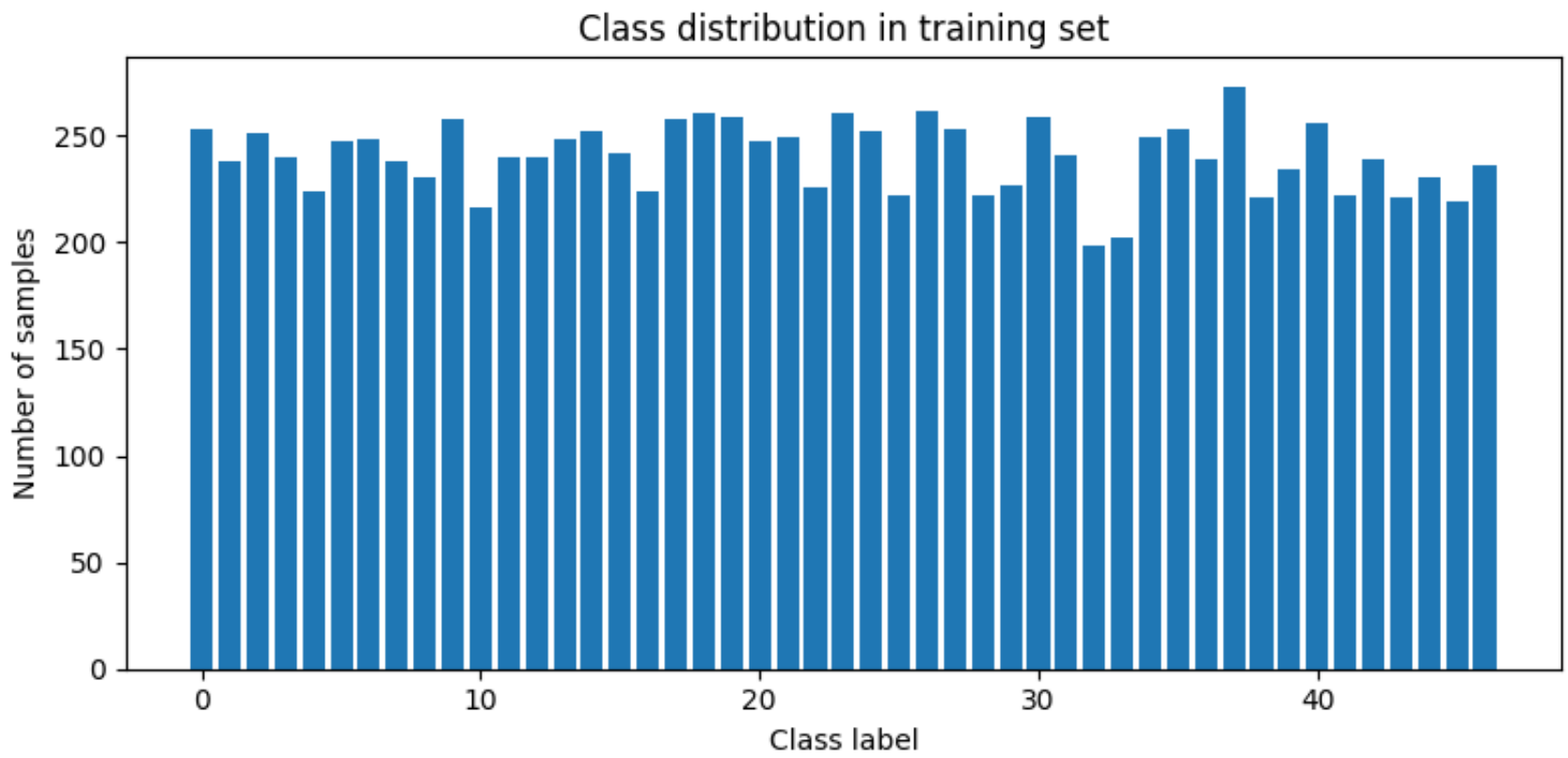
```python
def main():

    # Ensure the results directory exists and set up logging to file and console
    os.makedirs(RESULTS_DIR, exist_ok=True)
    log_path = os.path.join(RESULTS_DIR, "results_log.txt")

    with open(log_path, "w", encoding="utf-8") as log_file:
        # Use the Tee class to write output to both console and log file
        tee = Tee(sys.stdout, log_file)
        old_stdout = sys.stdout
        sys.stdout = tee
        try:
            # ============ Data loading ============
            train_X, train_Y, test_X, test_Y = load_data("image_data.npz")

            # Question 1(a): descriptive statistics and class distribution
            image_side, classes = describe_datasets(
                train_X, train_Y, test_X, test_Y
            )

            # Question 1(b): mean and median images for four randomly chosen classes
            selected_classes = plot_mean_median_images(
                train_X, train_Y, image_side, classes
            )

            # Question 1(c): 2D PCA visualisation of the selected two classes
            two_classes = selected_classes[:2]  # Use the first two classes for subsequent analysis
            visualise_two_classes_pca(train_X, train_Y, image_side, two_classes)

            # Question 1(d): within-class similarity vs random images
            label_for_subset = two_classes[0]  # Use the first class as the reference
            compare_similarity_within_class_vs_random(
                train_X, train_Y, label_for_subset
            )

            # ============ Question 2: classification and clustering on two selected classes ============
            print(
                "=== Question 2: Classification and clustering on two selected classes ==="
            )
            print("Selected classes:", two_classes)

            # Extract training and test subsets for the two classes
            mask_train_two = np.isin(train_Y, two_classes)
            X_train_two = train_X[mask_train_two]
            y_train_two = train_Y[mask_train_two]

            mask_test_two = np.isin(test_Y, two_classes)
            X_test_two = test_X[mask_test_two]
            y_test_two = test_Y[mask_test_two]

            print("Training subset shape:", X_train_two.shape)
            print("Test subset shape:", X_test_two.shape)

            # Scale features for models sensitive to feature magnitude (logistic regression, k-NN)
            scaler_base = StandardScaler()
```

```python
X_train_two_scaled = scaler_base.fit_transform(X_train_two)
X_test_two_scaled = scaler_base.transform(X_test_two)

# ============ Question 2(a)(i): baseline classifiers (no hyperparameter tuning) ============
print(
    "=== Question 2(a)(i): Baseline classifiers (no hyperparameter tuning) ==="
)
# Initialise three classifiers with default parameters
log_reg = LogisticRegression(max_iter=1000, random_state=RANDOM_STATE)
knn = KNeighborsClassifier()  # 默认 k=5
rf = RandomForestClassifier(random_state=RANDOM_STATE, n_jobs=-1)

classifiers = [
    ("Logistic regression", log_reg),
    ("k-NN", knn),
    ("Random forest", rf),
]

# Train and evaluate each classifier
for name, clf in classifiers:
    if name == "Random forest":
        # Random forest does not require feature scaling
        clf.fit(X_train_two, y_train_two)
        y_pred = clf.predict(X_test_two)
    else:
        # Logistic regression and k-NN require feature scaling
        clf.fit(X_train_two_scaled, y_train_two)
        y_pred = clf.predict(X_test_two_scaled)

    acc = accuracy_score(y_test_two, y_pred)
    print(f"{name} test accuracy (no tuning): {acc:.4f}")

# ============ Question 2(a)(ii): hyperparameter tuning with K-fold cross-validation ============
print(
    "=== Question 2(a)(ii): Hyperparameter tuning with K-fold cross-validation ==="
)

# --- Tune the C parameter of logistic regression ---
C_values = [0.01, 0.1, 1.0, 10.0, 100.0]
best_C, best_score_log = tune_logistic_regression(
    X_train_two, y_train_two, C_values
)
print(
    f"Best C for logistic regression: {best_C}, "
    f"mean CV accuracy={best_score_log:.4f}"
)

# Train on the full training set with the best parameter and evaluate on the test set
scaler_log_final = StandardScaler()
X_train_log_scaled = scaler_log_final.fit_transform(X_train_two)
X_test_log_scaled = scaler_log_final.transform(X_test_two)
log_reg_opt = LogisticRegression(
    C=best_C, max_iter=1000, random_state=RANDOM_STATE
)
log_reg_opt.fit(X_train_log_scaled, y_train_two)
y_test_pred_log_opt = log_reg_opt.predict(X_test_log_scaled)
acc_log_opt = accuracy_score(y_test_two, y_test_pred_log_opt)
print(f"Logistic regression (tuned) test accuracy: {acc_log_opt:.4f}")

# --- Tune the number of neighbours for k-NN ---
k_values = [1, 3, 5, 7, 9, 11]
best_k, best_score_knn = tune_knn_classifier(
    X_train_two, y_train_two, k_values
)
print(
    f"Best k for k-NN: {best_k}, mean CV accuracy={best_score_knn:.4f}"
)

scaler_knn_final = StandardScaler()
X_train_knn_scaled = scaler_knn_final.fit_transform(X_train_two)
X_test_knn_scaled = scaler_knn_final.transform(X_test_two)
knn_opt = KNeighborsClassifier(n_neighbors=best_k)
knn_opt.fit(X_train_knn_scaled, y_train_two)
y_test_pred_knn_opt = knn_opt.predict(X_test_knn_scaled)
acc_knn_opt = accuracy_score(y_test_two, y_test_pred_knn_opt)
print(f"k-NN (tuned) test accuracy: {acc_knn_opt:.4f}")

# --- Tune n_estimators and max_depth for random forest ---
n_estimators_list = [50, 100, 200]
```

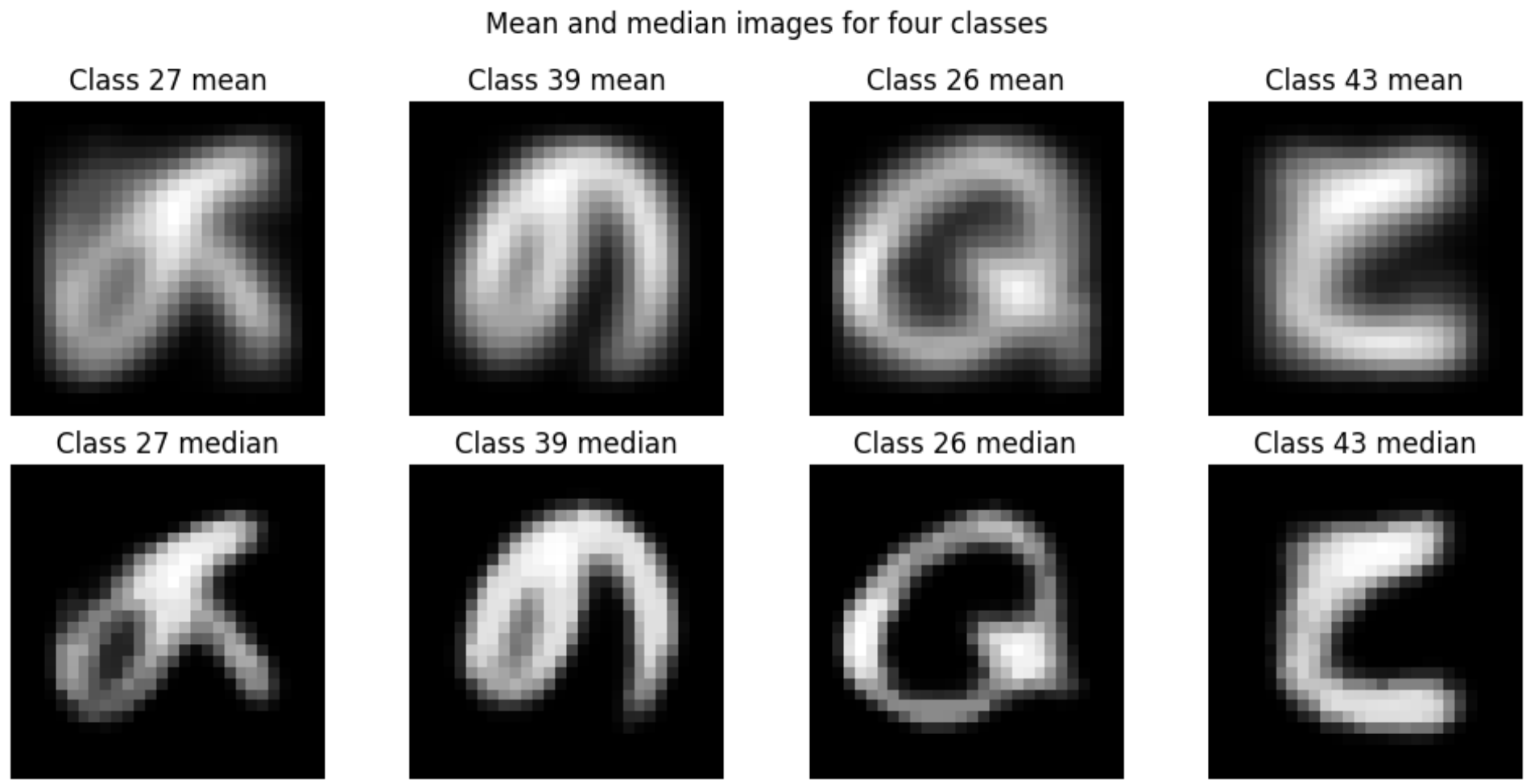
```python
max_depth_list = [None, 10, 20]  # None means trees can grow without a depth limit
best_rf_params, best_score_rf = tune_random_forest_classifier(
    X_train_two, y_train_two, n_estimators_list, max_depth_list
)
best_n_estimators, best_max_depth = best_rf_params
print(
    "Best RF params: "
    f"n_estimators={best_n_estimators}, max_depth={best_max_depth}, "
    f"mean CV accuracy={best_score_rf:.4f}"
)

rf_opt = RandomForestClassifier(
    n_estimators=best_n_estimators,
    max_depth=best_max_depth,
    random_state=RANDOM_STATE,
    n_jobs=-1,
)
rf_opt.fit(X_train_two, y_train_two)
y_test_pred_rf_opt = rf_opt.predict(X_test_two)
acc_rf_opt = accuracy_score(y_test_two, y_test_pred_rf_opt)
print(f"Random forest (tuned) test accuracy: {acc_rf_opt:.4f}")

# ============ Question 2(b): k-means clustering on the test subset ============
print("=== Question 2(b): k-means clustering on the test subset ===")
# Scale the test features (clustering algorithms are sensitive to feature scales)
scaler_kmeans = StandardScaler()
X_test_two_scaled_kmeans = scaler_kmeans.fit_transform(X_test_two)

# Run k-means clustering into 2 clusters (corresponding to the two classes)
kmeans = KMeans(n_clusters=2, random_state=RANDOM_STATE, n_init=10)
cluster_labels = kmeans.fit_predict(X_test_two_scaled_kmeans)

# Compute clustering purity to evaluate clustering quality
purity = cluster_purity(y_test_two, cluster_labels)
print(f"k-means cluster purity on test subset: {purity:.4f}")

# Print a contingency table: true labels vs cluster labels
print("Contingency table (true label vs cluster):")
for c in two_classes:
    for cluster in range(2):
        count = np.sum((y_test_two == c) & (cluster_labels == cluster))
        print(f"Class {c}, cluster {cluster}: {count}")

# Visualise clustering results in PCA-reduced space
pca_kmeans = PCA(n_components=2, random_state=RANDOM_STATE)
X_test_two_pca = pca_kmeans.fit_transform(X_test_two_scaled_kmeans)

# Figure 1: colored by cluster labels
plt.figure(figsize=(6, 6))
plt.scatter(
    X_test_two_pca[:, 0],
    X_test_two_pca[:, 1],
    c=cluster_labels,
    cmap="viridis",
    alpha=0.6,
    s=20,
)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("k-means clusters on test subset (colored by cluster)")
plt.tight_layout()
plt.savefig(
    os.path.join(RESULTS_DIR, "q2b_kmeans_clusters_by_cluster.png")
)
plt.show()
plt.close()

# Figure 2: colored by true labels for comparison
plt.figure(figsize=(6, 6))
markers = ["o", "s"]
for i, c in enumerate(two_classes):
    mask_c = y_test_two == c
    plt.scatter(
        X_test_two_pca[mask_c, 0],
        X_test_two_pca[mask_c, 1],
        marker=markers[i],
        alpha=0.6,
        s=20,
        label=f"Class {c}",
```

```
                )
            plt.xlabel("PC1")
            plt.ylabel("PC2")
            plt.title("Test subset in PCA space (colored by true label)")
            plt.legend()
            plt.tight_layout()
            plt.savefig(
                os.path.join(RESULTS_DIR, "q2b_test_pca_by_true_label.png")
            )
            plt.show()
            plt.close()
        finally:
            sys.stdout = old_stdout


if __name__ == "__main__":
    main()
```

```
=== Question 1(a): Descriptive statistics ===
Train set: 11280 samples, 784 features (28x28 images)
Test set:  1880 samples, 784 features (28x28 images)
Number of classes in training set: 47
```

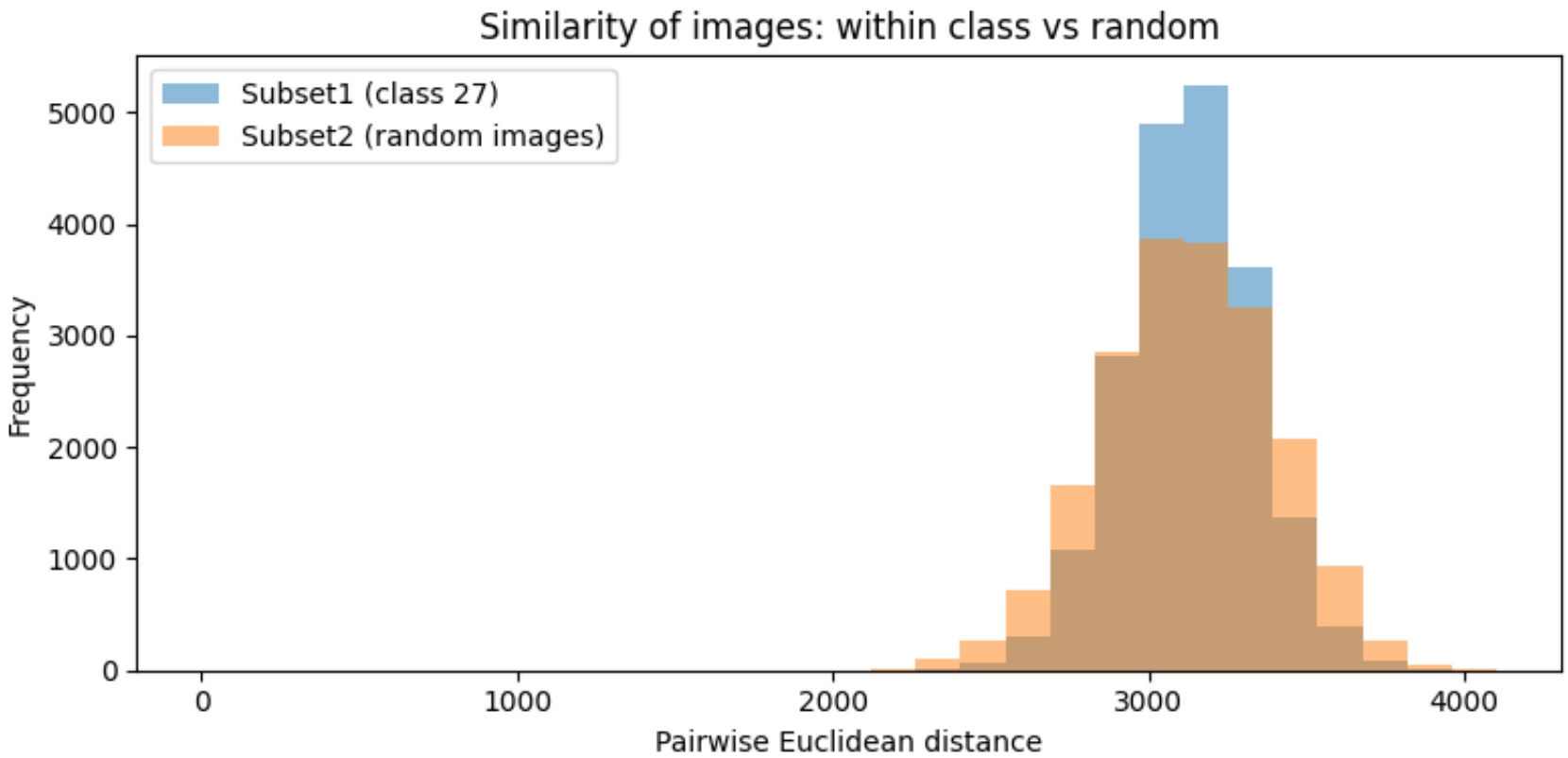



```
=== Question 1(b): Mean and median images ===
Selected classes for mean/median images: [27 39 26 43]
```

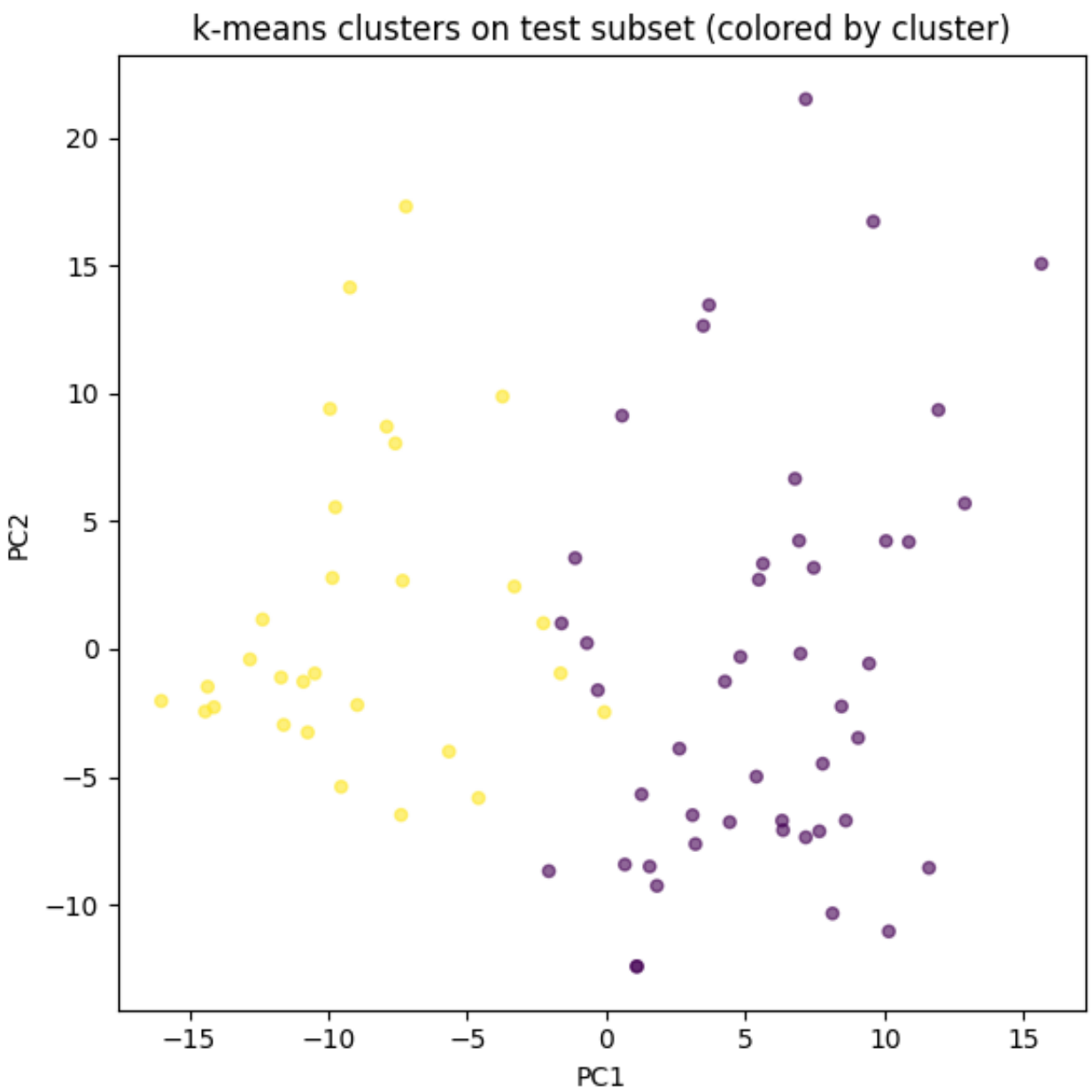


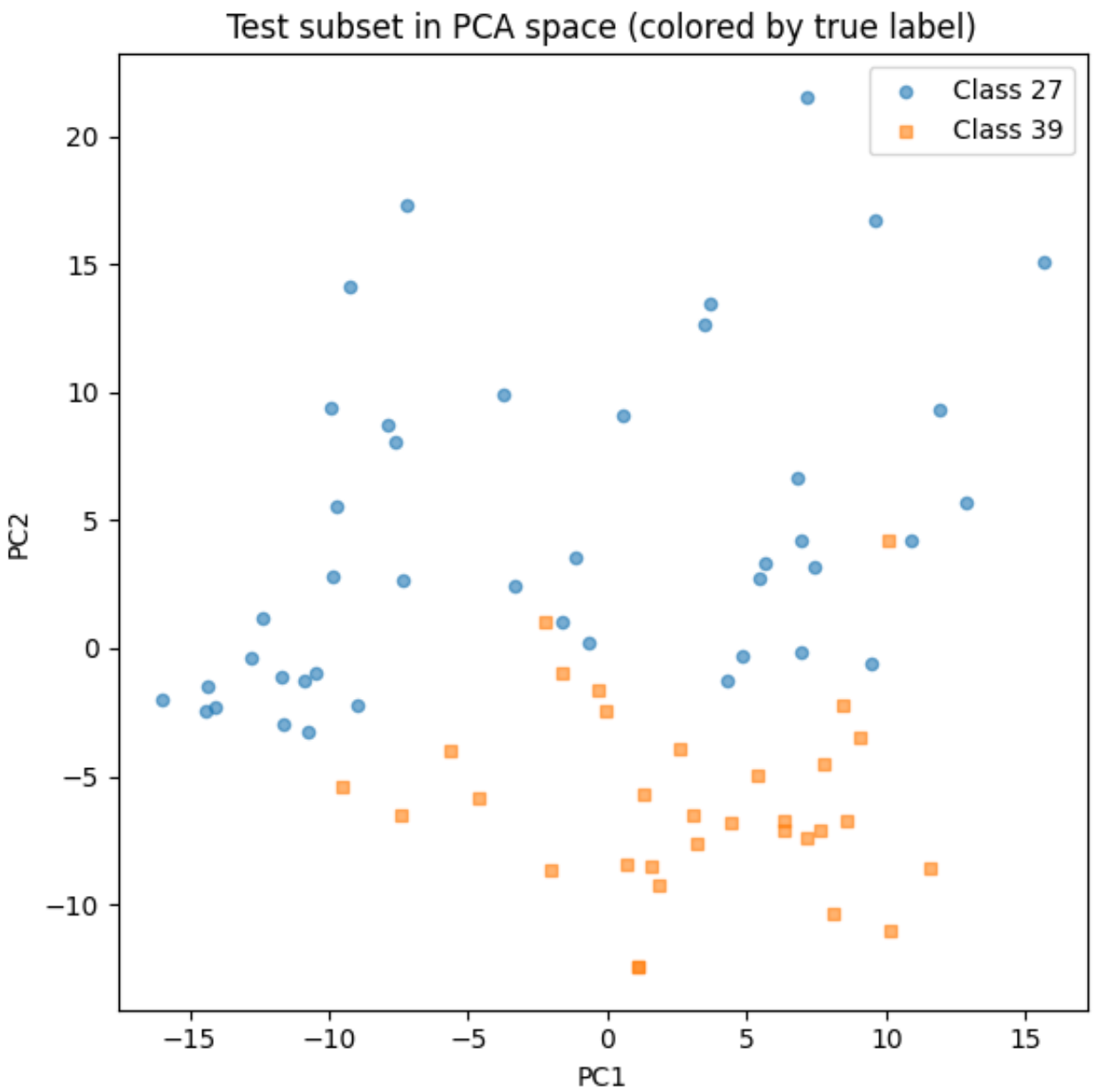

```
=== Question 1(c): 2D visualisation of two classes ===
Two classes used for 2D visualisation: [27 39]
```

```
=== Question 1(d): Similarity within class vs random images ===
Reference class for subset 1: 27
Subset1 (class 27) size: 200
Subset2 (random) size: 200
```



```
=== Question 2: Classification and clustering on two selected classes ===
Selected classes: [27 39]
Training subset shape: (487, 784)
Test subset shape: (75, 784)
=== Question 2(a)(i): Baseline classifiers (no hyperparameter tuning) ===
Logistic regression test accuracy (no tuning): 0.9733
k-NN test accuracy (no tuning): 0.9467
Random forest test accuracy (no tuning): 0.9600
=== Question 2(a)(ii): Hyperparameter tuning with K-fold cross-validation ===
Logistic regression: C=0.01 mean CV accuracy=0.9610
Logistic regression: C=0.1 mean CV accuracy=0.9589
Logistic regression: C=1.0 mean CV accuracy=0.9568
Logistic regression: C=10.0 mean CV accuracy=0.9548
Logistic regression: C=100.0 mean CV accuracy=0.9486
Best C for logistic regression: 0.01, mean CV accuracy=0.9610
Logistic regression (tuned) test accuracy: 0.9600
k-NN: k=1 mean CV accuracy=0.9527
k-NN: k=3 mean CV accuracy=0.9445
k-NN: k=5 mean CV accuracy=0.9486
k-NN: k=7 mean CV accuracy=0.9384
k-NN: k=9 mean CV accuracy=0.9302
k-NN: k=11 mean CV accuracy=0.9261
Best k for k-NN: 1, mean CV accuracy=0.9527
k-NN (tuned) test accuracy: 0.9600
Random forest: n_estimators=50, max_depth=None mean CV accuracy=0.9610
Random forest: n_estimators=50, max_depth=10 mean CV accuracy=0.9610
Random forest: n_estimators=50, max_depth=20 mean CV accuracy=0.9610
Random forest: n_estimators=100, max_depth=None mean CV accuracy=0.9631
Random forest: n_estimators=100, max_depth=10 mean CV accuracy=0.9631
Random forest: n_estimators=100, max_depth=20 mean CV accuracy=0.9631
Random forest: n_estimators=200, max_depth=None mean CV accuracy=0.9610
Random forest: n_estimators=200, max_depth=10 mean CV accuracy=0.9610
Random forest: n_estimators=200, max_depth=20 mean CV accuracy=0.9610
Best RF params: n_estimators=100, max_depth=None, mean CV accuracy=0.9631
Random forest (tuned) test accuracy: 0.9600
=== Question 2(b): k-means clustering on the test subset ===
k-means cluster purity on test subset: 0.6267
Contingency table (true label vs cluster):
Class 27, cluster 0: 21
Class 27, cluster 1: 22
Class 39, cluster 0: 25
Class 39, cluster 1: 7
```





PartB - Applied Statics

Code

Set up

In [178…

```
import os
import sys

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats
import statsmodels.api as sm
import statsmodels.formula.api as smf

#Fix random seeds to ensure reproducible results
RANDOM_STATE = 123
#Define the result storage directory and manage the result files in a unified manner
RESULTS_DIR = "results"
#Show pictures
SHOW_PLOTS = True

def save_and_show(fig, filename):
    fig.tight_layout()
    fig.savefig(os.path.join(RESULTS_DIR, filename))
    if SHOW_PLOTS:
        plt.show()
    plt.close(fig)



class Tee:
    """A simple tee class that writes stdout to both the console and a log file."""

    def __init__(self, *files):
        self.files = files

    def write(self, obj):
```

```
        for f in self.files:
            f.write(obj)

    def flush(self):
        for f in self.files:
            f.flush()
```

Question 3

Question 3(a):

In [179…

```
def question3a():
    rng = np.random.default_rng(RANDOM_STATE)

    # Common parameters
    n_counters = 100            # number of Geiger counters
    t_interval = 10.0           # seconds per observation in parts (a)-(d)
    r_rate = 0.4                # particle rate per second for all counters in parts (a)-(d)
    lam_per_counter = r_rate * t_interval

    print("\n=== Question 3(a): simulate Y with alpha = 1 ===")
    alpha = 1.0
    n_samples = 1000


    X_samples = rng.poisson(lam=lam_per_counter, size=(n_samples, n_counters))
    Y_samples = X_samples.sum(axis=1)

    print(f"Sample mean of Y (alpha=1): {Y_samples.mean():.2f}")
    print(f"Sample variance of Y (alpha=1): {Y_samples.var(ddof=1):.2f}")

    # Empirical pmf (histogram) and empirical CDF of Y
    fig, axes = plt.subplots(1, 2, figsize=(10, 4))

    # Histogram (empirical distribution of Y)
    axes[0].hist(Y_samples, bins=20, density=True, alpha=0.7, edgecolor="black")
    axes[0].set_xlabel("Y")
    axes[0].set_ylabel("Empirical density")
    axes[0].set_title("Empirical distribution of Y (alpha=1)")

    # Empirical CDF
    y_sorted = np.sort(Y_samples)
    ecdf = np.arange(1, n_samples + 1) / n_samples
    axes[1].step(y_sorted, ecdf, where="post")
    axes[1].set_xlabel("Y")
    axes[1].set_ylabel("Empirical CDF")
    axes[1].set_title("Empirical CDF of Y (alpha=1)")

    save_and_show(fig, "q3a_empirical_dist_cdf_Y.png")
```

Explanation:

- When (\alpha = 1), every count is transmitted successfully ((W_i = 1)).
- The count (X_i) of counter (i) within 10 seconds follows a Poisson distribution with mean (\lambda = r \times t = 0.4 \times 10 = 4).
- (Y) is the total detected count:
  (Y = \sum_{i=1}^{n} X_i).

Question 3(b)

In [180…

```
def question3b():
    rng = np.random.default_rng(RANDOM_STATE)

    n_counters = 100
    t_interval = 10.0
    r_rate = 0.4
    lam_per_counter = r_rate * t_interval
    print("\n=== Question 3(b): approximate simulation models for Y ===")

   # Model 1 (exact but simplified): by additivity of independent Poisson variables,
    # simulate Y directly as a single Poisson random variable.
    lam_Y = n_counters * lam_per_counter
    Y_poisson = rng.poisson(lam=lam_Y, size=n_samples)
```

```
    print(f"Model 1 (Poisson) mean: {Y_poisson.mean():.2f}")
    print(f"Model 1 (Poisson) variance: {Y_poisson.var(ddof=1):.2f}")

    # Model 2 (approximation): use a Normal approximation for Y. For a Poisson variable with mean lambda_Y,
    # the variance is also lambda_Y; by the CLT, Y is approximately Normal(lambda_Y, lambda_Y).
    Y_normal = rng.normal(loc=lam_Y, scale=np.sqrt(lam_Y), size=n_samples)
    # Y must be a count, so we round and truncate at zero.
    Y_normal = np.clip(np.round(Y_normal), a_min=0, a_max=None).astype(int)

    print(f"Model 2 (Normal approx.) mean: {Y_normal.mean():.2f}")
    print(f"Model 2 (Normal approx.) variance: {Y_normal.var(ddof=1):.2f}")

    # Compare empirical CDFs of the three models in a single plot
    fig, ax = plt.subplots(figsize=(6, 5))
    for data, label, style in [
        (Y_samples, "Full network (baseline)", "k-"),
        (Y_poisson, "Poisson(λ=400)", "r--"),
        (Y_normal, "Normal approximation", "b:"),
    ]:
        d_sorted = np.sort(data)
        ecdf = np.arange(1, len(d_sorted) + 1) / len(d_sorted)
        ax.step(d_sorted, ecdf, style, where="post", label=label)

    ax.set_xlabel("Y")
    ax.set_ylabel("Empirical CDF")
    ax.set_title("Q3(b): comparison of empirical CDFs of Y")
    ax.legend()
    save_and_show(fig, "q3b_empirical_cdfs_approx_models.png")
```

Explanation:

Model 1 (exact but simplified): by additivity of independent Poisson variables,

Model 2 (approximation): use a Normal approximation for Y. For a Poisson variable with mean lambda_Y,

Discussion :

```
- The Poisson model's empirical CDF is almost identical to the full network simulation,
  confirming that summing 100 independent Poisson(4) counters is equivalent to a single
Poisson(400) model.

- The Normal approximation is close in the bulk of the distribution but less accurate in the
tails and for integer-valued outcomes.
```

Question 3(c)

In [181…

```
def question3c():
    rng = np.random.default_rng(RANDOM_STATE)

    n_counters = 100
    t_interval = 10.0
    r_rate = 0.4
    lam_per_counter = r_rate * t_interval
    lam_Y = n_counters * lam_per_counter

    print("\n=== Question 3(c): one-sample t-test on simulated Y ===")

    # We test at the 5% level H0: mean(Y) = 400 vs H1: mean(Y) > 400.
    mu0 = lam_Y
    y_mean = Y_samples.mean()
    y_sd = Y_samples.std(ddof=1)
    t_stat = (y_mean - mu0) / (y_sd / np.sqrt(n_samples))
    df = n_samples - 1
    p_value = 1.0 - stats.t.cdf(t_stat, df=df)  # 单侧检验

    print(f"Sample mean of Y: {y_mean:.2f}")
    print(f"Sample standard deviation of Y: {y_sd:.2f}")
    print(f"t statistic: {t_stat:.3f}, df={df}")
    print(f"One-sided p-value for H0: mean(Y)=400 vs H1: mean(Y)>400: {p_value:.4f}")
```

Explanation:

Discussion:

```
- Because the data are generated under a model with true mean 400, the t-test will rarely
reject H0 at the 5% level; the p-value depends on the particular simulated sample.
```

```
- Hypothesis testing on purely simulated data is mainly used to check that the test behaves as
expected (here the Type I error rate should be about 5%)
```

Question 3(d):

In [182…

```python
def question3d():
    rng = np.random.default_rng(RANDOM_STATE)

    n_counters = 100
    t_interval = 10.0
    r_rate = 0.4
    lam_per_counter = r_rate * t_interval
```

Explanation:

No code required. Suggested discussion :

```
# - If alpha = 0.5 and the detection rate r is the same for all sensors,

#   each Wi is Bernoulli(0.5). The expected contribution of one counter

#   becomes 0.5 * E[Xi] = 0.5 * 4 = 2, so E[Y] drops from 400 to 200 and Var(Y) also decreases.

# - Model 1 (Poisson, with parameter equal to the sum of means) is still exact, but now
lambda_Y = 200.

# - The Normal approximation still uses mean 200 and variance 200, so its accuracy in (b)
should be similar,

#   just centred at a lower value.
```

Question 3(e):

In [183…

```python
def question3e():
    rng = np.random.default_rng(RANDOM_STATE)

    n_counters = 100
    t_interval = 10.0
    r_rate = 0.4
    lam_per_counter = r_rate * t_interval

    print("\n=== Question 3(e): mixture of counters with alpha = 0.5 ===")

    alpha = 0.5
    n_counters_low = 50
    n_counters_high = 50
    r_low = 0.4   # low-rate counters, per second
    r_high = 1.0  # high-rate counters, per second
    n_seconds = 1000

    reported_counts = []
    for _ in range(n_seconds):
        # Low-rate counters
        X_low = rng.poisson(lam=r_low, size=n_counters_low)
        W_low = rng.binomial(n=1, p=alpha, size=n_counters_low)
        reported_counts.extend(X_low[W_low == 1])

        # High-rate counters
        X_high = rng.poisson(lam=r_high, size=n_counters_high)
        W_high = rng.binomial(n=1, p=alpha, size=n_counters_high)
        reported_counts.extend(X_high[W_high == 1])

    reported_counts = np.array(reported_counts)
    print(f"Number of reported counts in dataset: {reported_counts.size}")
    print(f"Mean of reported counts: {reported_counts.mean():.3f}")
    print(f"Variance of reported counts: {reported_counts.var(ddof=1):.3f}")

    # Empirical distribution (histogram) and CDF
    fig, axes = plt.subplots(1, 2, figsize=(10, 4))

    axes[0].hist(
        reported_counts,
        bins=np.arange(reported_counts.max() + 2) - 0.5,
        density=True,
        edgecolor="black",
        alpha=0.7,
```

```
    )
    axes[0].set_xlabel("Reported count")
    axes[0].set_ylabel("Empirical probability")
    axes[0].set_title("Empirical distribution of reported counts")

    rc_sorted = np.sort(reported_counts)
    ecdf_rc = np.arange(1, len(rc_sorted) + 1) / len(rc_sorted)
    axes[1].step(rc_sorted, ecdf_rc, where="post")
    axes[1].set_xlabel("Reported count")
    axes[1].set_ylabel("Empirical CDF")
    axes[1].set_title("Empirical CDF of reported counts")

    save_and_show(fig, "q3e_reported_counts_dist_cdf.png")
```

Explanation:

We simulate per-second counts X_i for each counter.

Only successfully transmitted counts (Wi = 1) are included in the

"reported counts via the network" dataset. This creates a mixture of two sub-populations

(low-rate and high-rate counters), which is useful for part (f).

Question 3(f)

In [184…

```
def question3f():
    rng = np.random.default_rng(RANDOM_STATE)

    n_counters = 100
    t_interval = 10.0
    r_rate = 0.4
    lam_per_counter = r_rate * t_interval
    print("\n=== Question 3(f): fit 2-component Normal mixture by MLE ===")


    x = reported_counts.astype(float)

    # Helper function: Gaussian probability density function
    def normal_pdf(x, mu, sigma):
        return (
            1.0
            / (np.sqrt(2.0 * np.pi) * sigma)
            * np.exp(-0.5 * ((x - mu) / sigma) ** 2)
        )

    # EM initialisation
    rng_mix = np.random.default_rng(RANDOM_STATE)
    pi1 = 0.5
    mu1, mu2 = np.percentile(x, [25, 75])
    sigma1 = x.std(ddof=1) / 2.0
    sigma2 = x.std(ddof=1)

    log_likelihoods = []
    max_iter = 200
    tol = 1e-6

    for iteration in range(max_iter):
        # E-step: responsibilities for component 1
        f1 = pi1 * normal_pdf(x, mu1, sigma1)
        f2 = (1.0 - pi1) * normal_pdf(x, mu2, sigma2)
        denom = f1 + f2 + 1e-15
        gamma1 = f1 / denom
        gamma2 = 1.0 - gamma1

        # M-step: update parameters using weighted maximum likelihood formulas
        N1 = gamma1.sum()
        N2 = gamma2.sum()

        # Avoid degenerate components by enforcing a minimum mixing weight
        pi1_new = N1 / (N1 + N2)
        pi1_new = np.clip(pi1_new, 1e-3, 1.0 - 1e-3)

        mu1_new = (gamma1 * x).sum() / max(N1, 1e-6)
        mu2_new = (gamma2 * x).sum() / max(N2, 1e-6)

        sigma1_new = np.sqrt(
            ((gamma1 * (x - mu1_new) ** 2).sum()) / max(N1, 1e-6)
```

```
        )
        sigma2_new = np.sqrt(
            ((gamma2 * (x - mu2_new) ** 2).sum()) / max(N2, 1e-6)
        )
        # Enforce a minimum standard deviation to avoid division by zero
        sigma1_new = max(sigma1_new, 1e-3)
        sigma2_new = max(sigma2_new, 1e-3)

        # Compute log-likelihood to monitor convergence
        mixture_pdf = (
            pi1_new * normal_pdf(x, mu1_new, sigma1_new)
            + (1.0 - pi1_new) * normal_pdf(x, mu2_new, sigma2_new)
        )
        ll = np.sum(np.log(mixture_pdf + 1e-15))
        log_likelihoods.append(ll)

        # Check convergence
        if iteration > 0 and abs(ll - log_likelihoods[-2]) < tol:
            pi1, mu1, mu2, sigma1, sigma2 = (
                pi1_new,
                mu1_new,
                mu2_new,
                sigma1_new,
                sigma2_new,
            )
            break

    pi1, mu1, mu2, sigma1, sigma2 = (
        pi1_new,
        mu1_new,
        mu2_new,
        sigma1_new,
        sigma2_new,
    )

print(f"EM iterations: {len(log_likelihoods)}")
print(f"Estimated mixture weight pi1: {pi1:.3f}")
print(f"Estimated mu1, sigma1: {mu1:.3f}, {sigma1:.3f}")
print(f"Estimated mu2, sigma2: {mu2:.3f}, {sigma2:.3f}")

# Plot empirical histogram and fitted mixture density
fig, ax = plt.subplots(figsize=(6, 4))
bins = np.arange(x.max() + 2) - 0.5
ax.hist(x, bins=bins, density=True, alpha=0.6, edgecolor="black", label="Data")

grid = np.linspace(0, x.max() + 1, 300)
mixture_density = (
    pi1 * normal_pdf(grid, mu1, sigma1)
    + (1.0 - pi1) * normal_pdf(grid, mu2, sigma2)
)
ax.plot(grid, mixture_density, "r-", label="Fitted 2-Normal mixture")

ax.set_xlabel("Reported count")
ax.set_ylabel("Density")
ax.set_title("Q3(f): fitted 2-Normal mixture to reported counts")
ax.legend()
save_and_show(fig, "q3f_mixture_fit.png")
```

Explanation:

We use the EM algorithm to fit a Gaussian mixture model (two components) to the counts from part (e).

This provides maximum likelihood estimates of component means, variances and mixing weights.

Discussion:

-The estimated means should roughly correspond to the two rate groups (low- and high-rate counters), but the underlying data are discrete counts rather than continuous Normal variables.

-The Normal mixture may reasonably fit the middle of the distribution, but may misrepresent the probability of zero counts and tail behaviour. A Poisson mixture would be more appropriate for count data.

Question4

Question4 (a):

In [185…

```
def question4a():
    # -------------------- Q4(a) -------------------------------------
    print("\n=== Question 4(a): burning_experiment.txt analysis ===")

    # Load the data: 25 scented candle brands, with two ingredients and gas emissions in mg.
    data = pd.read_csv("burning_experiment.txt", sep=r"\s+")
    print("First rows of data:")
    print(data.head())

    # Basic descriptive statistics
    print("\nDescriptive statistics:")
    print(data.describe())

    # Scatter plots to visualise the relationships with gas emissions
    fig, axes = plt.subplots(1, 2, figsize=(10, 4))
    axes[0].scatter(data["ingredient1"], data["gas"])
    axes[0].set_xlabel("Ingredient 1 (mg)")
    axes[0].set_ylabel("Gas emission (mg)")
    axes[0].set_title("Gas vs ingredient 1")

    axes[1].scatter(data["ingredient2"], data["gas"])
    axes[1].set_xlabel("Ingredient 2 (mg)")
    axes[1].set_ylabel("Gas emission (mg)")
    axes[1].set_title("Gas vs ingredient 2")

    save_and_show(fig, "q4a_scatter_gas_vs_ingredients.png")


    # Fit a multiple linear regression model:
    #   gas = beta0 + beta1 * ingredient1 + beta2 * ingredient2 + error
    model = smf.ols("gas ~ ingredient1 + ingredient2", data=data).fit()
    print("\nOLS regression summary for gas ~ ingredient1 + ingredient2:")
    print(model.summary())

    # Fitted values and residuals for diagnostics
    fitted = model.fittedvalues
    residuals = model.resid

    # Diagnostic plots: residuals vs fitted, and normal Q-Q plot
    fig, axes = plt.subplots(1, 2, figsize=(10, 4))

    axes[0].scatter(fitted, residuals)
    axes[0].axhline(0, color="red", linestyle="--")
    axes[0].set_xlabel("Fitted values")
    axes[0].set_ylabel("Residuals")
    axes[0].set_title("Residuals vs fitted")

    sm.qqplot(residuals, line="45", ax=axes[1])
    axes[1].set_title("Normal Q-Q plot of residuals")

    save_and_show(fig, "q4a_residual_diagnostics.png")

    # Example prediction: gas emissions for a new candle at given ingredient levels (chosen within the observed
    new_candle = pd.DataFrame({"ingredient1": [12.0], "ingredient2": [0.9]})
    prediction = model.get_prediction(new_candle)
    print("\nPrediction for new candle (ingredient1=12, ingredient2=0.9):")
    print(prediction.summary_frame(alpha=0.05))
```

Explanation:

- Both ingredients show positive associations with gas emissions in the scatter plots, suggesting that increasing either ingredient increases emitted gas.
- The multiple regression model quantifies the separate effect of each ingredient while adjusting for the other.
- The regression summary allows us to comment on the magnitude and significance of each coefficient and overall model fit (R-squared, residual standard error).
- The residual and Q-Q plots help assess linearity, homoscedasticity and approximate normality of residuals.
- To improve the experiment, we could increase sample size, randomise ingredient combinations, repeat burns under the same formulation, and extend the range of ingredient amounts to better explore possible non-linear effects.

Question4 (b-i):

In [186…

```
def question4b_i():
    print("\n=== Question 4(b)(i): Gamma-type model with reciprocal(y) ===")

    rng = np.random.default_rng(RANDOM_STATE)
    n = 1000
```

```
# Toy data: simulate predictors x and z
x = rng.uniform(0, 5, size=n)
z = rng.choice([0, 1], size=n)  # for simplicity, z is binary

# True model for 1 / y (reciprocal response), roughly matching
# the reported coefficients: 2 (intercept), 1.5 (x), 0.6 (z), 0.5 (interaction).
eta = 2.0 + 1.5 * x + 0.6 * z + 0.5 * x * z
# Add small noise and take the reciprocal to obtain y > 0
eta_noisy = eta + rng.normal(0, 0.2, size=n)
y = 1.0 / eta_noisy

df_gamma = pd.DataFrame({"y": y, "x": x, "z": z})

# Fit a linear model with 1/y as the response to mimic the reported analysis
# (for simplicity, we use OLS on 1/y).
df_gamma["inv_y"] = 1.0 / df_gamma["y"]
lm_inv = smf.ols("inv_y ~ x * z", data=df_gamma).fit()
print(lm_inv.summary())

# Use the fitted model to visualise how y changes with x when z is fixed at 1.
# This shows that even if the effect is linear on 1/y, it is non-linear on y.
x_grid = np.linspace(0, 5, 100)
z_fixed = 1
inv_y_pred = lm_inv.predict(pd.DataFrame({"x": x_grid, "z": z_fixed}))
y_pred = 1.0 / inv_y_pred

fig, ax = plt.subplots(figsize=(6, 4))
ax.scatter(x[z == 1], y[z == 1], alpha=0.3, label="Simulated data (z=1)")
ax.plot(x_grid, y_pred, "r", label="Fitted relationship (z=1)")
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_title("Q4(b)(i): effect of x on y when z = 1")
ax.legend()
save_and_show(fig, "q4b_i_reciprocal_model.png")
```

Explanation:

Key discussion points:

- The reported coefficients refer to a model with 1/y as the response, so statements like "when x increases by one unit, y increases by 1.5 units" are incorrect.
- The relationship between x and y is non-linear even if 1/y is linear in x and z; the fitted curve above is clearly curved.
- The report should clarify the response scale (here 1/y), the link/function, and how to interpret effects on the original y scale.

Question4 (b-ii):

In [187…

```
def question4b_ii():
    print("\n=== Question 4(b)(ii): linear model with heteroscedastic residuals ===")

    # Toy data with a single predictor and strong heteroscedasticity.
    n2 = 1000
    x2 = np.linspace(0, 1, n2)
    # True non-linear mean and increasing variance
    mean_true = 0.05 + 0.05 * np.sin(4 * np.pi * x2)
    sd_true = 0.02 + 0.15 * (x2 ** 2)   # variance increases with x
    y2 = mean_true + rng.normal(0, sd_true)

    df_lm = pd.DataFrame({"x": x2, "y": y2})
    lm = smf.ols("y ~ x", data=df_lm).fit()
    print(lm.summary())

    fitted2 = lm.fittedvalues
    residuals2 = lm.resid

    fig, ax = plt.subplots(figsize=(6, 4))
    ax.scatter(fitted2, residuals2, s=10, alpha=0.5)
    ax.axhline(0, color="red", linestyle="--")
    ax.set_xlabel("Fitted values")
    ax.set_ylabel("Residuals")
    ax.set_title("Q4(b)(ii): residuals vs fitted (toy data)")
    save_and_show(fig, "q4b_ii_residuals_vs_fitted.png")
```

Explanation:

Key discussion points:

- The residual plot shows a clear funnel shape and curvature, indicating strong heteroscedasticity and model misspecification.
- With 1000 data points, this pattern is not due to sampling noise and should not be ignored.
- A better analysis could use variance-stabilising transformations, non-linear models, or models that explicitly allow changing variance.

Question4 (b-iii)

In [188…

```
def question4b_iii():
    print("\n=== Question 4(b)(iii): randomisation test for two coins ===")

    # Toy data: two biased coins A and B, with observed proportions similar to the example.
    nA = 20
    nB = 40
    # Example counts: 16 heads out of 20 for A (0.8), 24 heads out of 40 for B (0.6)
    heads_A = 16
    heads_B = 24

    data_A = np.array([1] * heads_A + [0] * (nA - heads_A))
    data_B = np.array([1] * heads_B + [0] * (nB - heads_B))

    # Observed difference in proportions
    pA = data_A.mean()
    pB = data_B.mean()
    diff_obs = pA - pB
    print(f"Observed pA={pA:.2f}, pB={pB:.2f}, difference={diff_obs:.2f}")

    # A randomisation (permutation) test for equality of proportions.
    # H0: the two coins have the same probability of heads.
    n_perm = 500
    pooled = np.concatenate([data_A, data_B])
    diffs = []
    for _ in range(n_perm):
        rng.shuffle(pooled)
        perm_A = pooled[:nA]
        perm_B = pooled[nA:]
        diffs.append(perm_A.mean() - perm_B.mean())
    diffs = np.array(diffs)

    # Two-sided p-value (fraction of permuted differences whose absolute value is at least as extreme as observe
    p_perm = np.mean(np.abs(diffs) >= abs(diff_obs))
    print(f"Randomisation-test p-value (two-sided, {n_perm} perms): {p_perm:.3f}")

    # Plot the permutation distribution and mark the observed difference
    fig, ax = plt.subplots(figsize=(6, 4))
    ax.hist(diffs, bins=20, edgecolor="black", alpha=0.7)
    ax.axvline(diff_obs, color="red", linestyle="--", label="Observed diff")
    ax.set_xlabel("Permuted difference in proportions (pA - pB)")
    ax.set_ylabel("Frequency")
    ax.set_title("Q4(b)(iii): randomisation test for two coins")
    ax.legend()
    save_and_show(fig, "q4b_iii_randomisation_test.png")
```

Explanation:

Key discussion points:

The permutation distribution represents the differences we would see if the two coins had the same probability of heads, i.e., under H0.

The p-value quantifies how extreme the observed difference is relative to this distribution; if it is greater than 0.1, we have little evidence that the coins differ.

The report should clearly state the null hypothesis, the test statistic, and how the p-value is obtained from the permutation distribution.

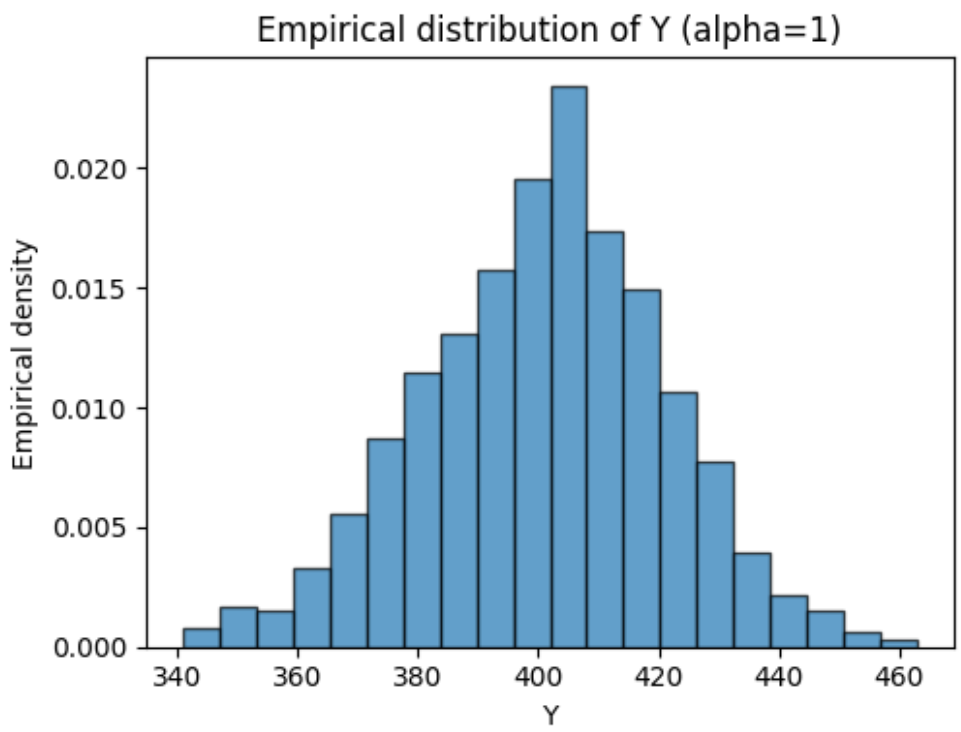
Answer:

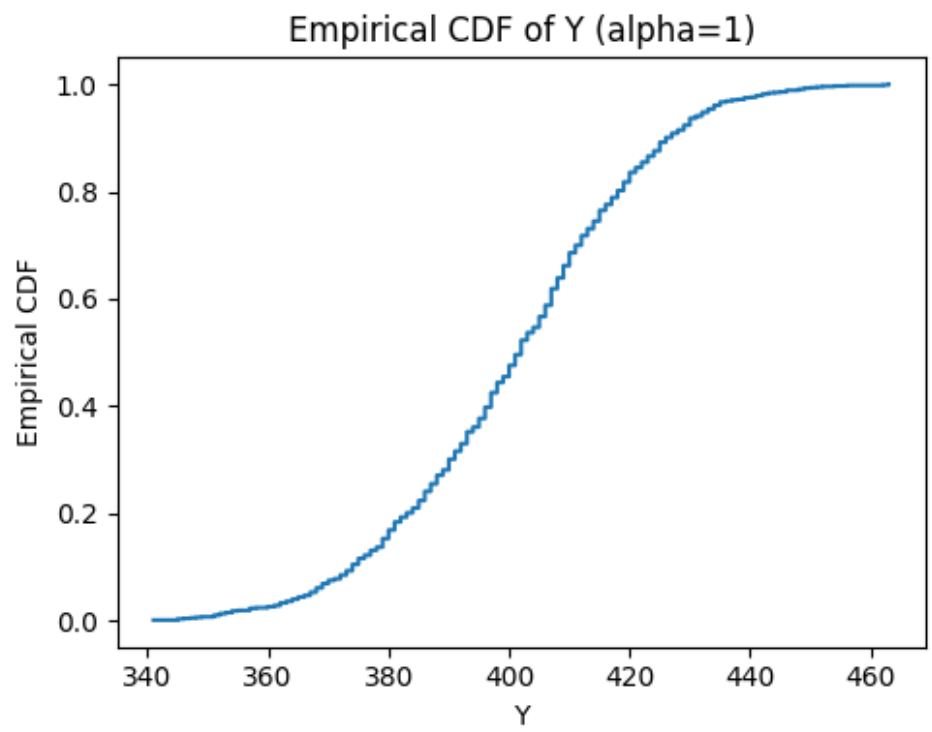
In [189…

```
def main():
    os.makedirs(RESULTS_DIR, exist_ok=True)
    log_path = os.path.join(RESULTS_DIR, "part2_q3_q4_log.txt")

    with open(log_path, "w", encoding="utf-8") as log_file:
        tee = Tee(sys.stdout, log_file)
```

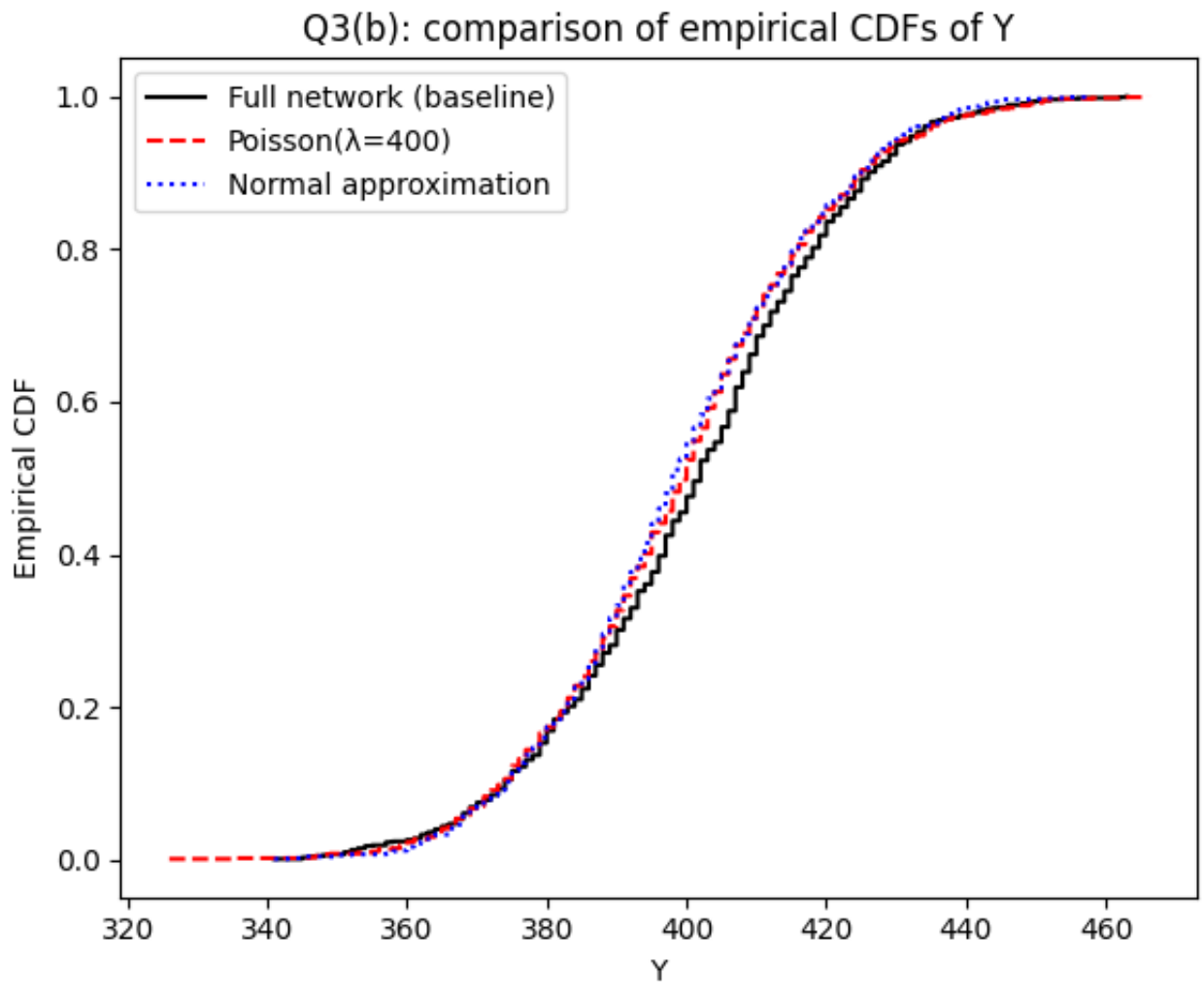
```
        old_stdout = sys.stdout
        sys.stdout = tee
        try:
            question3a()
            question3b()
            question3c()
            question3d()
            question3e()
            question3f()
            question4a()
            question4b_i()
            question4b_ii()
            question4b_iii()
        finally:
            sys.stdout = old_stdout


if __name__ == "__main__":
    main()
```

```
=== Question 3(a): simulate Y with alpha = 1 ===
Sample mean of Y (alpha=1): 400.87
Sample variance of Y (alpha=1): 414.37
```

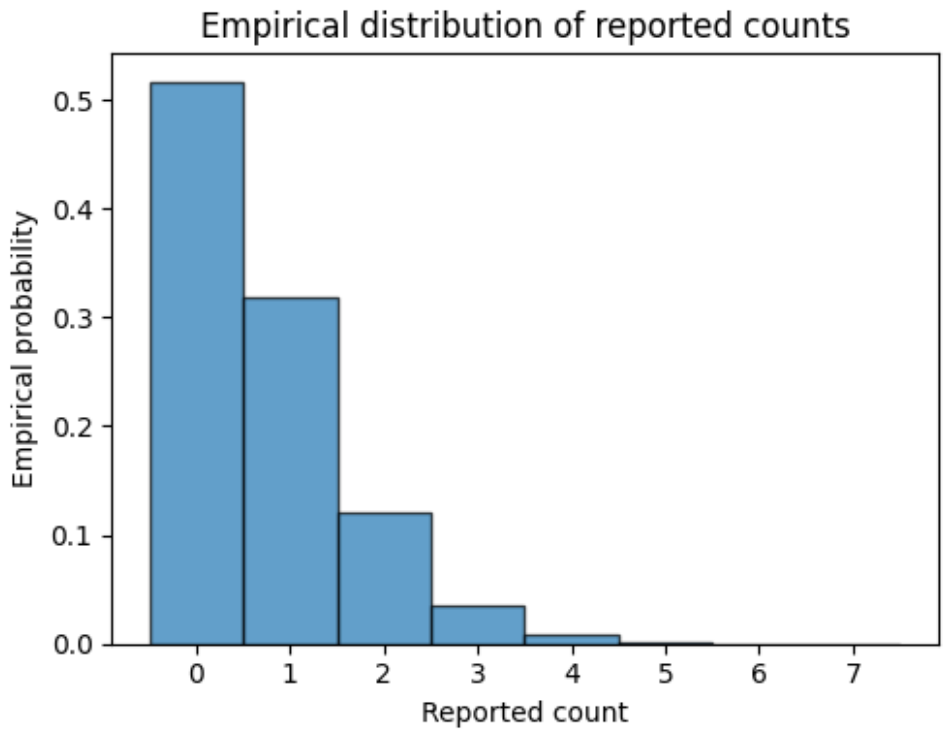


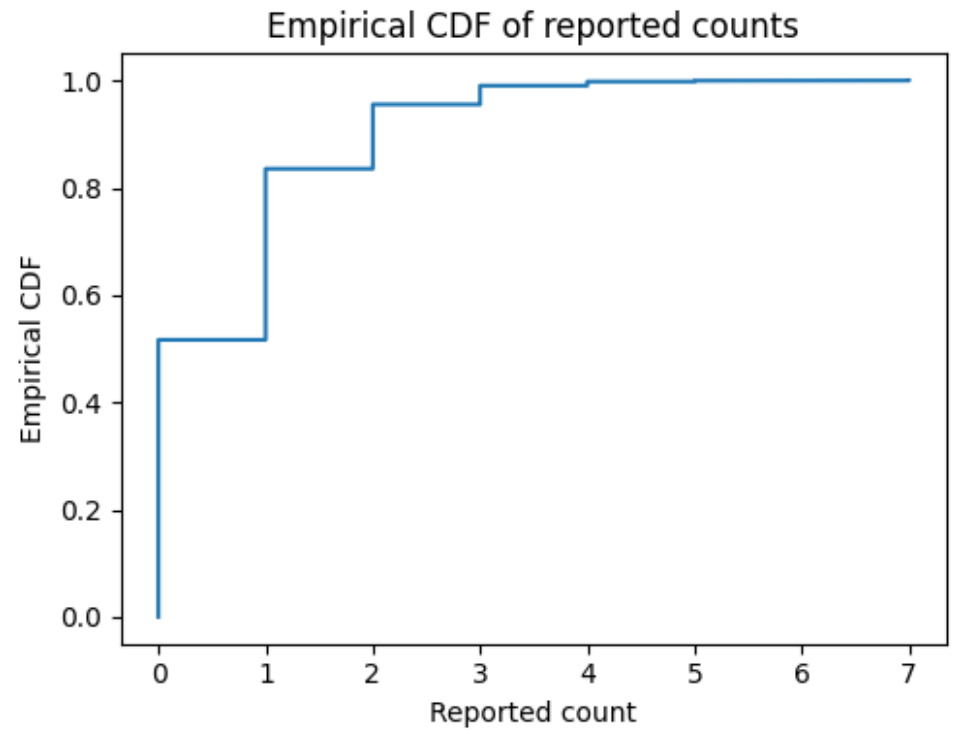



```
=== Question 3(b): approximate simulation models for Y ===
Model 1 (Poisson) mean: 399.37
Model 1 (Poisson) variance: 403.33
Model 2 (Normal approx.) mean: 399.03
Model 2 (Normal approx.) variance: 372.14
```

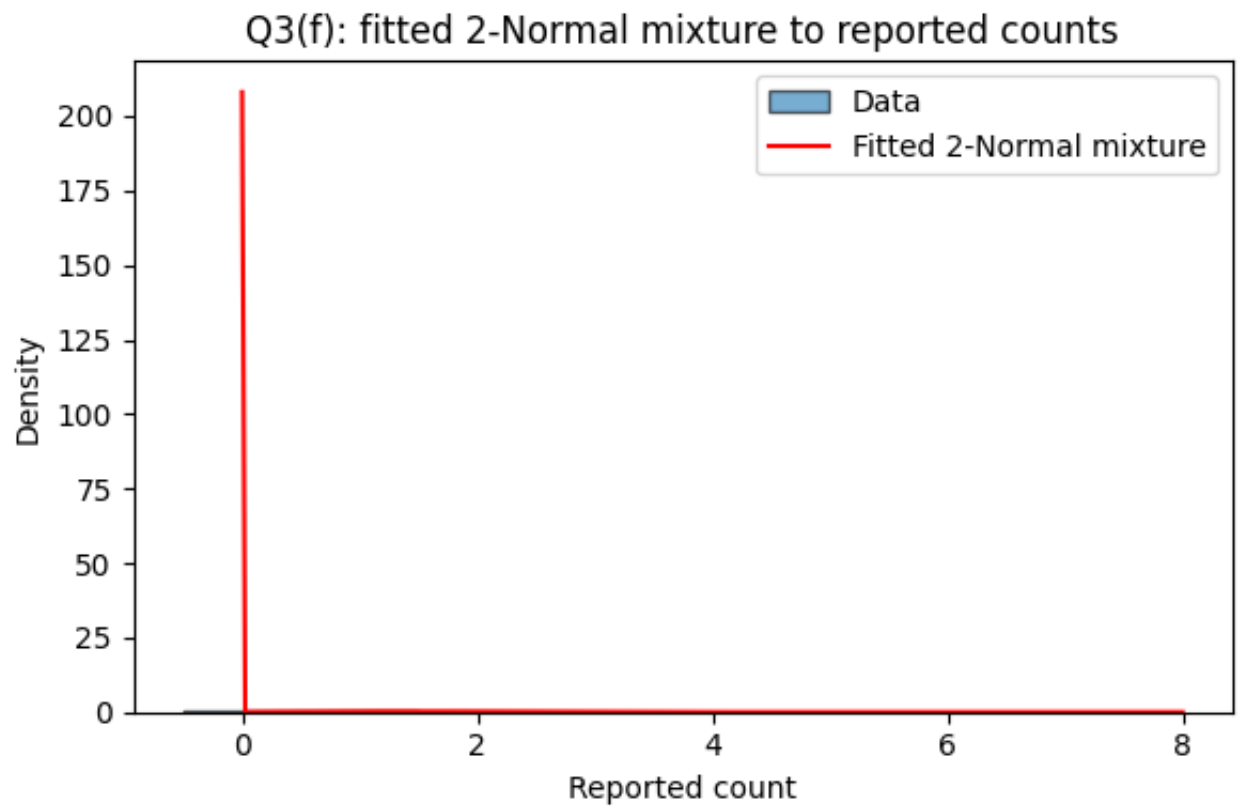


```
=== Question 3(c): one-sample t-test on simulated Y ===
Sample mean of Y: 400.87
Sample standard deviation of Y: 20.36
t statistic: 1.344, df=999
One-sided p-value for H0: mean(Y)=400 vs H1: mean(Y)>400: 0.0897

=== Question 3(e): mixture of counters with alpha = 0.5 ===
Number of reported counts in dataset: 49899
Mean of reported counts: 0.705
Variance of reported counts: 0.794
```

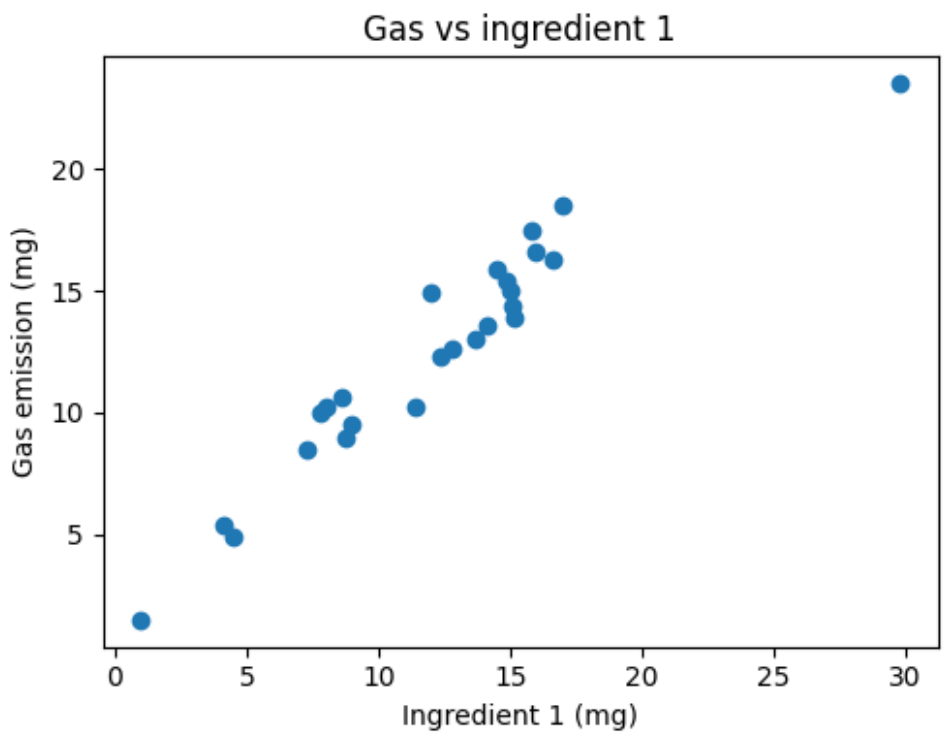


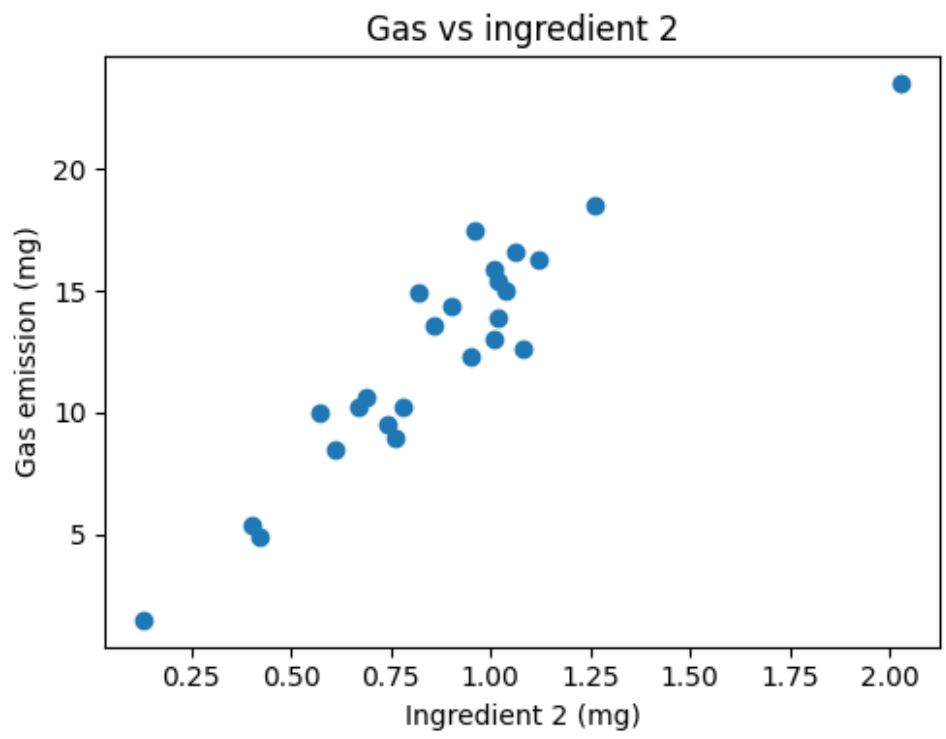



```
=== Question 3(f): fit 2-component Normal mixture by MLE ===
EM iterations: 6
Estimated mixture weight pi1: 0.521
Estimated mu1, sigma1: 0.000, 0.001
Estimated mu2, sigma2: 1.455, 0.734
```

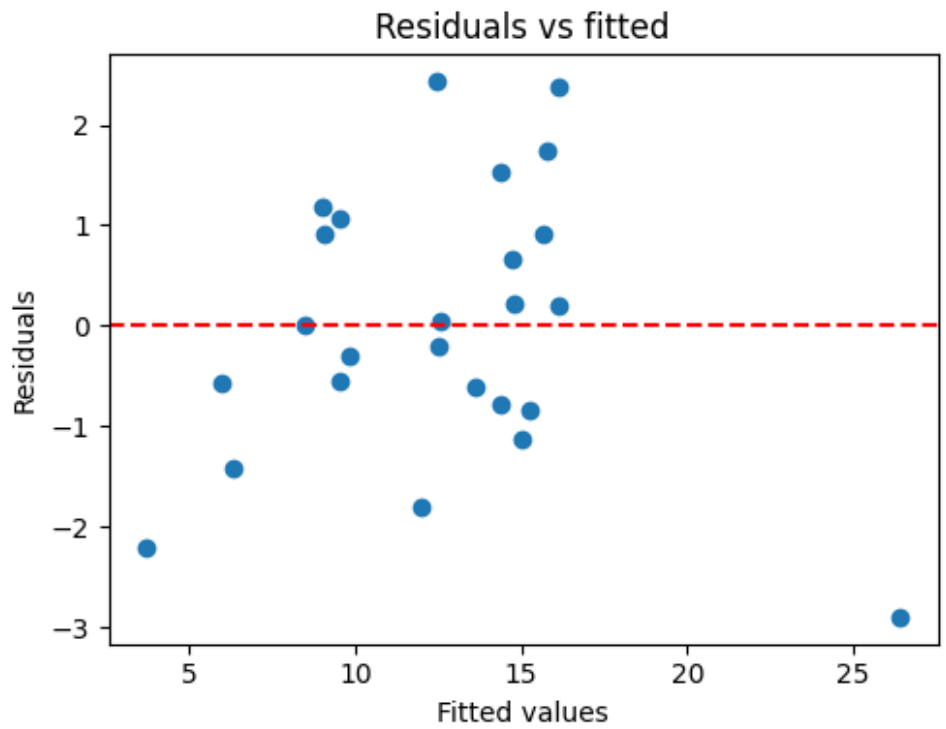


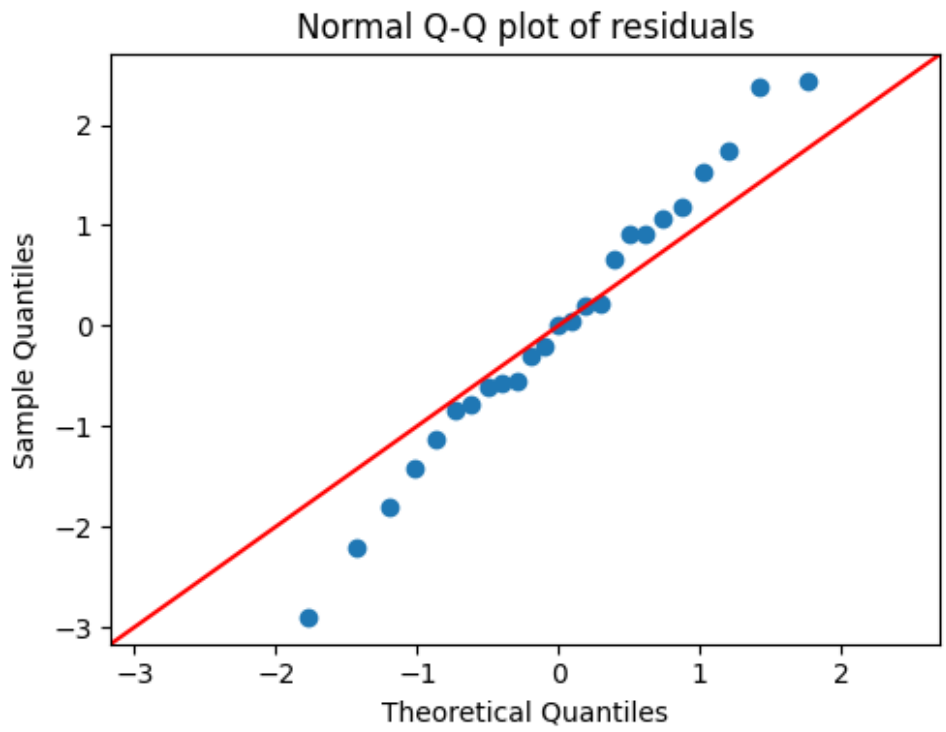

```
=== Question 4(a): burning_experiment.txt analysis ===
First rows of data:
   ingredient1  ingredient2   gas
0         14.1         0.86  13.6
1         16.0         1.06  16.6
2         29.8         2.03  23.5
3          8.0         0.67  10.2
4          4.1         0.40   5.4

Descriptive statistics:
       ingredient1  ingredient2        gas
count     25.00000    25.000000  25.000000
mean      12.21600     0.876400  12.528000
std        5.66581     0.354058   4.739684
min        1.00000     0.130000   1.500000
25%        8.60000     0.690000  10.000000
50%       12.80000     0.900000  13.000000
75%       15.10000     1.020000  15.400000
max       29.80000     2.030000  23.500000
```



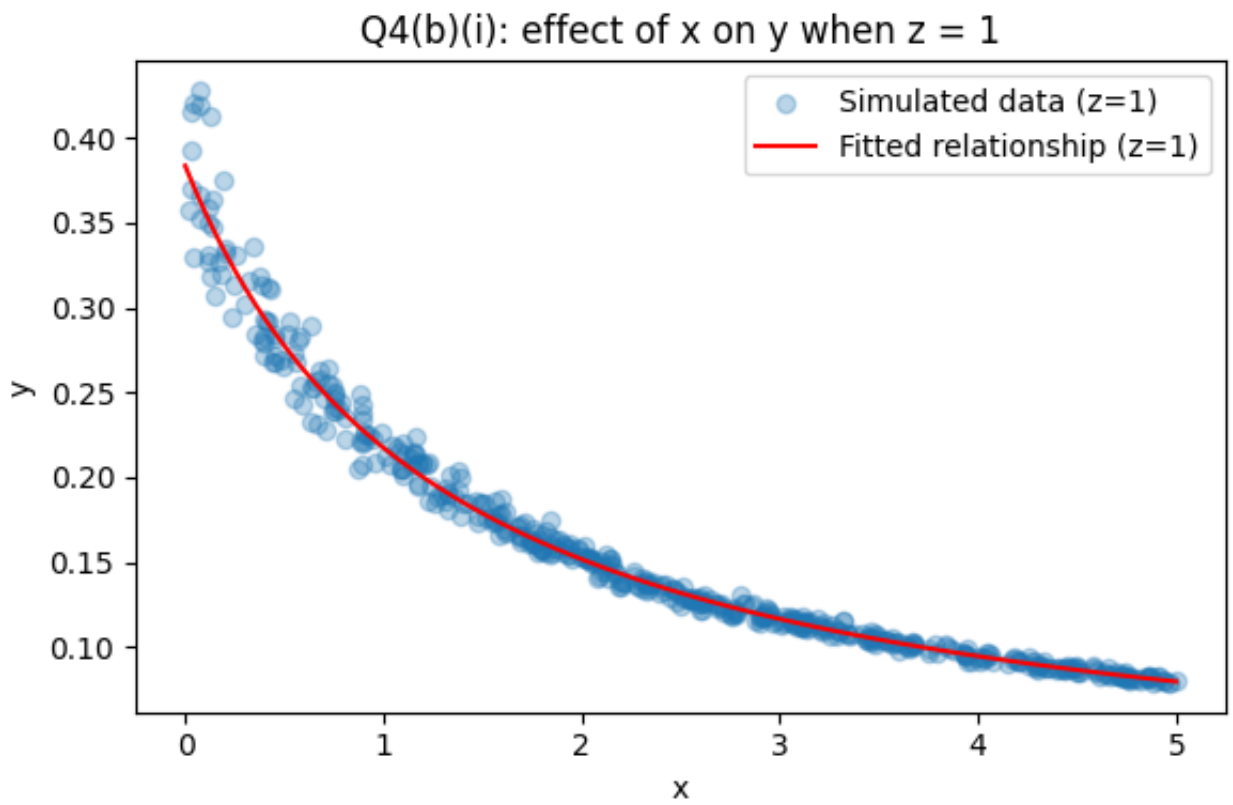



```
OLS regression summary for gas ~ ingredient1 + ingredient2:
                            OLS Regression Results
==============================================================================
Dep. Variable:                    gas   R-squared:                       0.919
Model:                            OLS   Adj. R-squared:                  0.911
Method:                 Least Squares   F-statistic:                     124.1
Date:                Mon, 08 Dec 2025   Prob (F-statistic):           1.04e-12
Time:                        15:27:58   Log-Likelihood:                -42.510
No. Observations:                  25   AIC:                             91.02
Df Residuals:                      22   BIC:                             94.68
Df Model:                           2
Covariance Type:            nonrobust
==================================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
----------------------------------------------------------------------------------
Intercept      3.0896      0.844      3.662      0.001       1.340       4.839
ingredient1    0.9625      0.237      4.067      0.001       0.472       1.453
ingredient2   -2.6463      3.787     -0.699      0.492     -10.500       5.208
==============================================================================
Omnibus:                        0.058   Durbin-Watson:                   2.865
Prob(Omnibus):                  0.972   Jarque-Bera (JB):                0.226
Skew:                          -0.093   Prob(JB):                        0.893
Kurtosis:                       2.572   Cond. No.                         183.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```

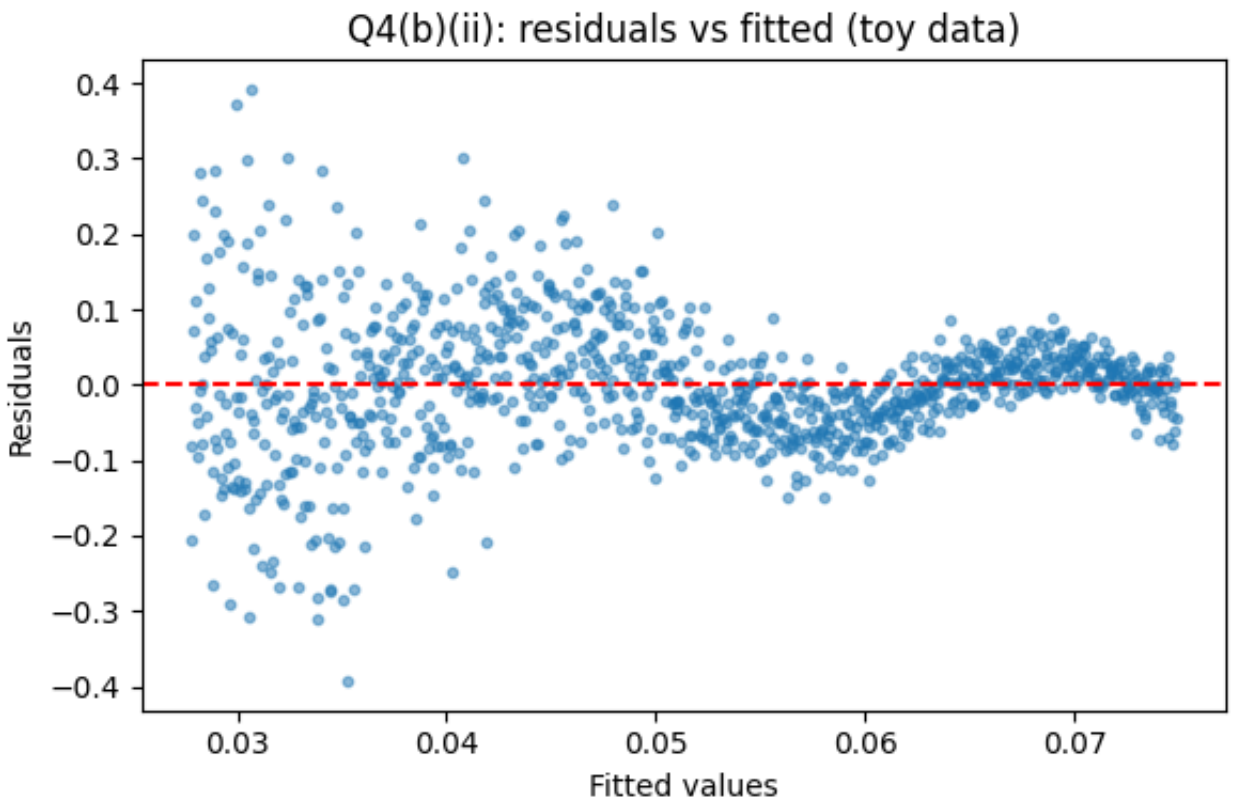


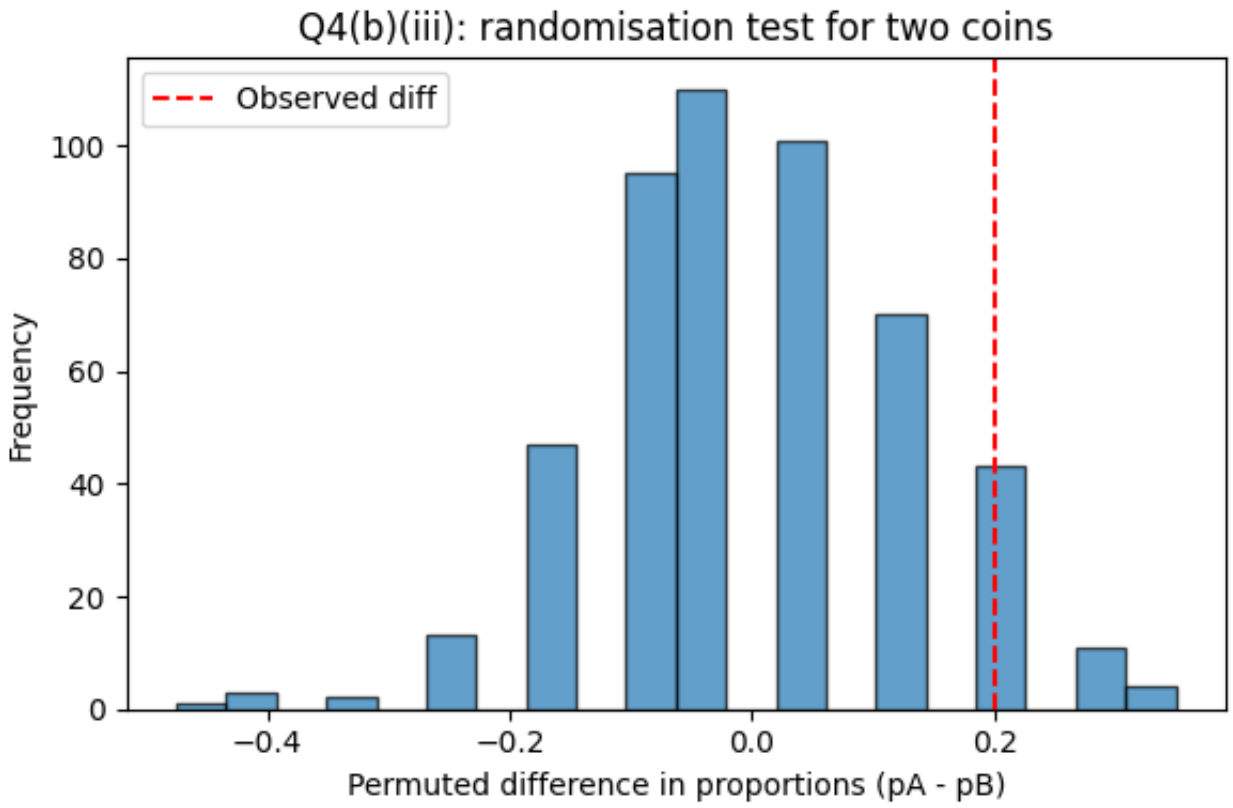


```
Prediction for new candle (ingredient1=12, ingredient2=0.9):
        mean   mean_se  mean_ci_lower  mean_ci_upper  obs_ci_lower  \
0  12.257654  0.315174      11.604023      12.911285      9.256222

   obs_ci_upper
0     15.259086

=== Question 4(b)(i): Gamma-type model with reciprocal(y) ===
                            OLS Regression Results
==============================================================================
Dep. Variable:                  inv_y   R-squared:                       0.994
Model:                            OLS   Adj. R-squared:                  0.994
Method:                 Least Squares   F-statistic:                 5.677e+04
Date:                Mon, 08 Dec 2025   Prob (F-statistic):               0.00
Time:                        15:27:58   Log-Likelihood:                 190.96
No. Observations:                1000   AIC:                            -373.9
Df Residuals:                     996   BIC:                            -354.3
Df Model:                           3
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept      1.9909      0.018    107.726      0.000       1.955       2.027
x              1.5051      0.006    242.874      0.000       1.493       1.517
z              0.6164      0.026     23.904      0.000       0.566       0.667
x:z            0.4882      0.009     54.722      0.000       0.471       0.506
==============================================================================
Omnibus:                        0.475   Durbin-Watson:                   1.972
Prob(Omnibus):                  0.789   Jarque-Bera (JB):                0.534
Skew:                          -0.049   Prob(JB):                        0.766
Kurtosis:                       2.942   Cond. No.                         16.8
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```



```
=== Question 4(b)(ii): linear model with heteroscedastic residuals ===
                            OLS Regression Results
==============================================================================
Dep. Variable:                      y   R-squared:                       0.024
Model:                            OLS   Adj. R-squared:                  0.023
Method:                 Least Squares   F-statistic:                     24.67
Date:                Mon, 08 Dec 2025   Prob (F-statistic):           7.99e-07
Time:                        15:27:58   Log-Likelihood:                 1028.2
No. Observations:                1000   AIC:                            -2052.
Df Residuals:                     998   BIC:                            -2043.
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept      0.0749      0.005     13.678      0.000       0.064       0.086
x             -0.0471      0.009     -4.967      0.000      -0.066      -0.028
==============================================================================
Omnibus:                       64.209   Durbin-Watson:                   1.801
Prob(Omnibus):                  0.000   Jarque-Bera (JB):              264.079
Skew:                           0.066   Prob(JB):                     4.53e-58
Kurtosis:                       5.514   Cond. No.                         4.39
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```




```
=== Question 4(b)(iii): randomisation test for two coins ===
Observed pA=0.80, pB=0.60, difference=0.20
Randomisation-test p-value (two-sided, 500 perms): 0.154
```